Parallel Transport of Tangent Vectors on Triangle Meshes: from the Vector Heat Equation to Code, with Finite Elements

Mauricio Cele Lopez Belon

Abstract. We derive, step by step, a program that spreads a tangent vector given at one point of a triangle mesh to every other point, so that the result at each point is the parallel transport of the given vector. The method is the vector heat method: diffuse the vector by the heat equation for tangent vector fields, whose operator is the connection Laplacian, and keep only the direction of the result. We start from the partial differential equation and go all the way down to the linear system that the code solves: the covariant derivative and the Levi-Civita connection of a surface, Green's identity for tangent fields, the backward Euler step, the weak form, the finite element space of piecewise linear tangent fields, the discrete parallel transport that makes that space possible, the gradients of the shape functions on a triangle embedded in space, the element stiffness and mass matrices, their assembly, the boundary condition, and the solution. Every step is motivated, every formula is derived, and every formula that the code evaluates is matched with the function that evaluates it, so that the text can be read side by side with the implementation [7]. A second part derives the same method in geometric algebra, where the tangent plane is a bivector, coordinates are complex numbers of that plane, parallel transport is a rolling rotor and the connection Laplacian becomes a complex Hermitian matrix, and it clarifies which parts of the computation are intrinsic to the curved surface.

Mathematics Subject Classification (2010). 65N30 Finite element methods; 53A05 Surfaces in Euclidean space; 65F05 Direct methods for linear systems.

Keywords. Finite Element Method, Connection Laplacian, Levi-Civita Connection, Parallel Transport, Vector Heat Method.

1. Introduction

Suppose we are given one tangent vector at one point of a curved surface, and we want to know what that vector "is" at every other point: the vector we would obtain by carrying it along the surface without turning it. On a plane the answer is trivial, the same vector everywhere. On a curved surface it is not, for two reasons. First, the tangent planes at different points are different planes, so the vector cannot simply be copied; it has to be carried, and the rule that says how is the *parallel transport* of the surface, given by its *Levi-Civita connection*. Second, the result depends on the path: carrying a vector around a closed loop on a sphere brings it back rotated, by an angle equal to the curvature enclosed. A sensible global answer is to carry the vector along the shortest path from the source, the geodesic.

The *vector heat method* of Sharp, Soliman and Crane [9] computes that answer without computing a single geodesic. Its idea is simple. Heat spreading from a point source travels, for a short time, mostly along shortest paths. If what spreads is not a temperature but a tangent vector, which diffuses by the heat equation for tangent vector fields, then the direction of the diffused vector at each point is the direction of the source vector carried along the shortest path; only its length decays with distance. So: diffuse the vector for a short time, and normalize.

This text derives a finite element implementation of that idea [7], following the classical presentations of the finite element method [5, 8] and extending them from scalar functions on flat domains to tangent vector fields on surfaces. The derivation has a fixed structure, which recurs in every finite element method and which we follow strictly:

1. the continuous problem: the operator, the equation, the boundary condition (Sections 2–4);
2. the weak form, which needs only first derivatives (Section 5);
3. a finite dimensional space of candidate solutions, the *trial space*, and of *test functions* (Section 6);
4. the element matrices, computed triangle by triangle (Sections 7–8);
5. the assembly of the global matrices, the boundary condition, and the solution of the linear system (Sections 9–10).

The ingredient that is not in the scalar method is the connection: tangent vectors at different vertices live in different planes and cannot be added before they are brought into a common plane. Section 3 shows how the code does that, and Section 6 shows that this is exactly where the Levi-Civita connection enters the finite element method. Table 1 at the end of Part 1 lists every formula with the function of `FEM.cpp` that evaluates it. Part 2 repeats the derivation in the language of geometric algebra, where frames become a single vector, coordinates become complex numbers of the tangent plane, transports become rotors, and the gradients of the shape functions become a reciprocal frame; it also separates what is intrinsic to the curved surface from what depends on its embedding in space.

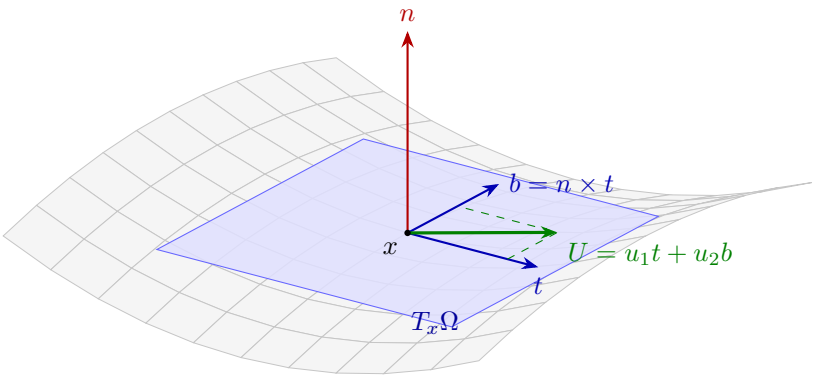


FIGURE 1. A point x of a surface, its normal n, its tangent plane $T_x\Omega$ (blue), an orthonormal frame (t, b) of the tangent plane, and a tangent vector U with its two coordinates u_1, u_2 in that frame.

Part 1. The Derivation with Matrices

2. Tangent Vectors on a Surface

2.1. Tangent planes and frames

Let $\Omega \subset \mathbb{R}^3$ be a smooth surface with unit normal $n(x)$ at each point x. The *tangent plane* $T_x\Omega$ is the plane through the origin orthogonal to $n(x)$, and a *tangent vector* at x is a vector of $\mathbb{R}^3$ in that plane. A *tangent vector field* U assigns to every point x a tangent vector $U(x) \in T_x\Omega$.

A vector of $\mathbb{R}^3$ needs three numbers, but a tangent vector needs only two, once we choose a basis of the tangent plane. We choose at each point an orthonormal, positively oriented *frame* $(t(x), b(x))$: two unit tangent vectors, orthogonal to each other, with $b = n \times t$. Collecting them as the columns of a 3×2 matrix,

$$T(x) = \begin{bmatrix} t(x) & b(x) \end{bmatrix}, \qquad T^T T = I_2, \qquad TT^T = I_3 - nn^T, \tag{2.1}$$

a tangent vector $U(x)$ has the coordinates $u = T^T U \in \mathbb{R}^2$, and is recovered as $U = Tu$. The two identities say that the columns are orthonormal, and that TT^T is the orthogonal projection of $\mathbb{R}^3$ onto the tangent plane. Since the frame is orthonormal, coordinates preserve lengths and angles: $U \cdot V = u \cdot v$ for tangent vectors at the same point.

The choice of t is arbitrary: any rotation of the frame in the tangent plane is as good. The physics does not depend on that choice, and we will see that the method does not either, because every quantity that compares vectors at different points will carry the change of frames with it.

2.2. The discrete surface

The surface is a triangle mesh with vertices $x_1, \ldots, x_n$ and triangular faces. For a face f with vertices $x_{f_1}, x_{f_2}, x_{f_3}$ in counterclockwise order, its unit

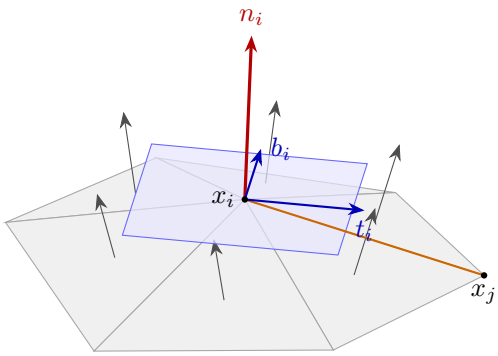


FIGURE 2. The one-ring of a vertex x_i. Each face has its own normal (gray arrows); the vertex normal n_i (red) is their area-weighted average (2.3). The vertex frame vector t_i is the edge to a neighbor x_j (orange), projected onto the vertex tangent plane (blue) and normalized, and $b_i = n_i \times t_i$.

normal and area are

$$n_f = \frac{(x_{f_2} - x_{f_1}) \times (x_{f_3} - x_{f_1})}{|(x_{f_2} - x_{f_1}) \times (x_{f_3} - x_{f_1})|}, \qquad A_f = \tfrac{1}{2}|(x_{f_2} - x_{f_1}) \times (x_{f_3} - x_{f_1})|, \quad (2.2)$$

which is `computeFaceNormals`. A face is flat: all its points share the tangent plane orthogonal to n_f. A vertex has no tangent plane of its own, since it is a corner shared by several faces, so we must choose one. The code takes the plane orthogonal to the *vertex normal*, the normalized sum of the unnormalized normals of the incident faces (`Mesh::computeNormals`), which weighs each face by its area:

$$n_i = \frac{\sum_{f \ni i} 2A_f \, n_f}{\left| \sum_{f \ni i} 2A_f \, n_f \right|}. \quad (2.3)$$

Frames. Each vertex and each face gets an orthonormal frame:

- *vertex* i (`computeVertexTangentSpace`): t_i is the projection, onto the tangent plane of i, of the edge from x_i to one of its neighbors x_j, normalized; and $b_i = n_i \times t_i$:

$$t_i = \frac{(x_j - x_i) - \big((x_j - x_i) \cdot n_i\big)\, n_i}{\big|(x_j - x_i) - \big((x_j - x_i) \cdot n_i\big)\, n_i\big|}, \qquad T_i = \big[\, t_i \;\; n_i \times t_i \,\big]; \quad (2.4)$$

- *face* f (`computeFaceTangentSpace`): t_f is the direction of its first edge, and $T_f = [\, t_f \;\; n_f \times t_f \,]$.

Unknowns. The unknown field is represented by one tangent vector per vertex, stored by its two coordinates in the vertex frame: $u_i \in \mathbb{R}^2$, with the vector $T_i u_i \in \mathbb{R}^3$. All the unknowns form one vector of $2n$ numbers,

$$\mathrm{u} = (u_{1,1}, u_{1,2}, u_{2,1}, u_{2,2}, \ldots, u_{n,1}, u_{n,2}), \quad (2.5)$$

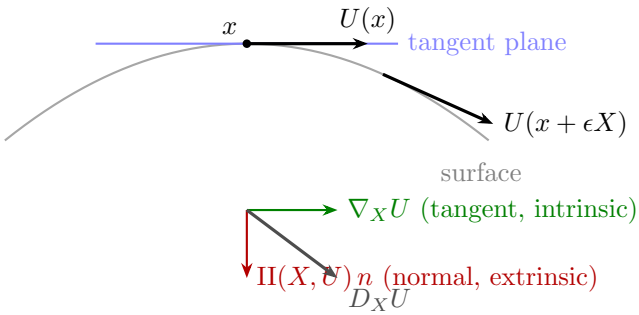


FIGURE 3. The covariant derivative. U at two nearby points lies in two different tangent planes; its ambient derivative $D_X U$ has a tangent part $\nabla_X U$, the covariant derivative, and a normal part, which depends only on how the surface bends (Section 14).

so that the coordinate $k \in \{1, 2\}$ of vertex i is the entry $2(i-1)+k$, or $2i+k$ with the zero-based indices of the code (`v->ID * 2 + row`). It is often convenient to read u_i as the complex number $u_{i,1} + \mathrm{i}\, u_{i,2}$: a rotation of the tangent plane by an angle α is then multiplication by $e^{\mathrm{i}\alpha}$. The code uses this reading to set the source vector and to draw the solution (Section 10).

3. Parallel Transport and the Levi-Civita Connection

3.1. The covariant derivative

To differentiate a tangent field we would like to compare $U(x + \epsilon X)$ with $U(x)$, but they lie in different tangent planes, and their difference is not a tangent vector. Differentiating in $\mathbb{R}^3$ is still possible: the *directional derivative* $D_X U$ of the three components of U in a tangent direction X is a vector of $\mathbb{R}^3$. It has a tangent part, which measures how U turns within the surface, and a normal part, which measures only how the surface itself bends under U. The *covariant derivative* keeps the tangent part:

$$\nabla_X U = \left(I - nn^T\right) D_X U = TT^T D_X U. \tag{3.1}$$

Figure 3 shows the split in a section of the surface: the tangent part measures the turning of U within the surface, the normal part only the bending of the surface.

This is the *Levi-Civita connection* of a surface in $\mathbb{R}^3$ [1]. Two of its properties will be used repeatedly. It is *compatible with the metric*: for tangent fields U, V,

$$X(U \cdot V) = \nabla_X U \cdot V + U \cdot \nabla_X V, \tag{3.2}$$

because $X(U \cdot V) = D_X U \cdot V + U \cdot D_X V$, and the normal parts of $D_X U$ and $D_X V$ are orthogonal to the tangent vectors V and U. And it is linear in X,

so at each point it defines a linear map $\nabla U : X \mapsto \nabla_X U$ from the tangent plane to itself, the *total covariant derivative*, a 2×2 matrix in a frame.
In a frame. Write $U = Tu$ with a frame field T. Then $D_X U = (D_X T)\, u + T\,(X \cdot \nabla u)$, and projecting with TT^T,

$$T^T \nabla_X U = X \cdot \nabla u + \omega(X)\, u, \qquad \omega(X) = T^T D_X T. \tag{3.3}$$

The first term is the ordinary derivative of the coordinates; the second corrects for the turning of the frame. The 2×2 matrix $\omega(X)$ is skew-symmetric, because differentiating $T^T T = I$ gives $\omega + \omega^T = 0$: it is $\begin{pmatrix} 0 & -\theta(X) \\ \theta(X) & 0 \end{pmatrix}$, an infinitesimal rotation by the angle $\theta(X)$, the *connection form* of the frame [6]. On a flat piece of surface with a constant frame, $\omega = 0$, and the covariant derivative is the ordinary derivative of the coordinates. This is the situation inside one face of a mesh, and it is what will make the finite element computation simple.

3.2. Parallel transport

A field U is *parallel* along a curve γ if it does not change along it in the sense of the connection, $\nabla_{\gamma'} U = 0$. Given U at the start of the curve, this linear differential equation determines it along the whole curve: that is the parallel transport of the vector along γ. By (3.2), $\frac{d}{ds}(U \cdot V) = 0$ for two parallel fields, so parallel transport preserves lengths and angles: it is a rotation from one tangent plane onto another.

On a mesh we need it only between a vertex and an incident face, two points whose tangent planes, with normals n_i and n_f, are close. Of all the rotations of $\mathbb{R}^3$ that take n_i to n_f, and therefore the tangent plane of i onto that of f, the natural one is the *smallest*: the rotation about the axis $n_i \times n_f$ by the angle between the normals. It is the discrete analogue of the transport along the short path from the vertex into the face. When the two planes are those of two faces sharing an edge, the axis $n_f \times n_g$ is the direction of the shared edge, and this rotation is exactly the *unfolding* of one face onto the plane of the other about their common edge, which is how a vector is transported across an edge of a polyhedral surface. (Figure 4.) The smallest rotation is given by the Rodrigues formula.

Proposition 3.1. *Let a and c be unit vectors with $a \cdot c \neq -1$, and let $v = a \times c$ and $[v]_\times$ the matrix of $w \mapsto v \times w$. Then*

$$Q = I + [v]_\times + \frac{1}{1 + a \cdot c}\, [v]_\times^2 \tag{3.4}$$

is the rotation about the axis v that takes a to c.

Proof. Let ϕ be the angle between a and c, so that $a \cdot c = \cos\phi$ and $|v| = \sin\phi$, and let $k = v/|v|$ be the unit axis. The Rodrigues rotation by ϕ about k is $I + \sin\phi\, [k]_\times + (1 - \cos\phi)\, [k]_\times^2$. Since $[v]_\times = \sin\phi\, [k]_\times$ and $[v]_\times^2 = \sin^2\phi\, [k]_\times^2$, and $\frac{\sin^2\phi}{1+\cos\phi} = \frac{1-\cos^2\phi}{1+\cos\phi} = 1 - \cos\phi$, this is (3.4). It takes a to c because a is orthogonal to the axis k and the angle from a to c about k is ϕ. □

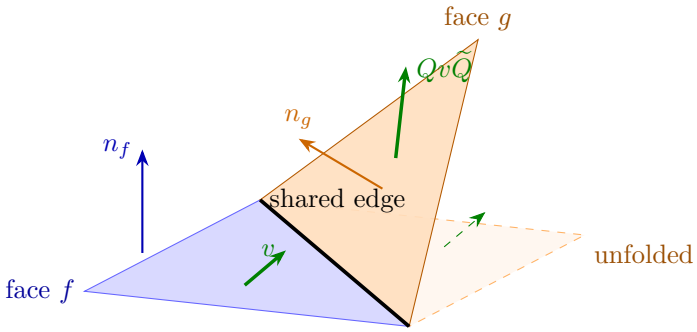


FIGURE 4. Transport across an edge. Two faces sharing an edge meet at an angle in space; unfolding face g into the plane of face f (dashed) is a rotation about the shared edge, and that rotation is exactly the smallest rotation taking n_f to n_g. The vector v of face f and its transport $Qv\widetilde{Q}$ into face g are the same vector of the unfolded plane.

This is `alignVectors`(n_i, n_f), with $[v]_\times$ computed by `bracket`. Figure 5 shows the rotation and the planes it connects. The formula fails when the normals are opposite, $a \cdot c = -1$, where the axis is undetermined; the code then returns $-I$, which only occurs for degenerate meshes (a face folded over onto its vertex) and is not a rotation.

The discrete connection. The rotation $Q_{i\to f}$ takes tangent vectors of vertex i to tangent vectors of face f. In coordinates, a tangent vector $T_i u$ of i becomes $Q_{i\to f} T_i u$, whose coordinates in the face frame are $T_f^T Q_{i\to f} T_i u$. So the transport from vertex i to face f, in coordinates, is the 2×2 matrix

$$R_i^f = T_f^T \, Q_{i\to f} \, T_i, \tag{3.5}$$

which is `Rvf`(v_i, f).

Proposition 3.2. *R_i^f is a rotation of $\mathbb{R}^2$: $(R_i^f)^T R_i^f = I_2$ and $\det R_i^f = 1$.*

Proof. The columns of T_i are orthonormal and orthogonal to n_i, so those of $Q_{i\to f} T_i$ are orthonormal and orthogonal to $Q_{i\to f}\, n_i = n_f$: they are an orthonormal basis of the tangent plane of f. The columns of T_f are another one, so T_f^T, restricted to that plane, is an isometry onto $\mathbb{R}^2$, and R_i^f, the matrix of a change between two orthonormal bases of the same plane, is orthogonal. Both bases are positively oriented with respect to n_f: T_f by construction, and $Q_{i\to f} T_i$ because the rotation Q preserves orientation and takes $n_i \times t_i = b_i$ to $n_f \times Q t_i$. So the determinant is $+1$. □

Worked example 3.3. *Let the vertex have the normal $n_i = (0, 0, 1)$ and the frame vector $t_i = (0, 1, 0)$, so $b_i = n_i \times t_i = (-1, 0, 0)$, and let the face have the normal $n_f = (0, -1, 1)/\sqrt{2}$ and the frame vector $t_f = (1, 0, 0)$, so $b_f = n_f \times t_f = (0, 1, 1)/\sqrt{2}$ (this is the face of Example 7.2). Then $n_i \cdot n_f = 1/\sqrt{2}$*

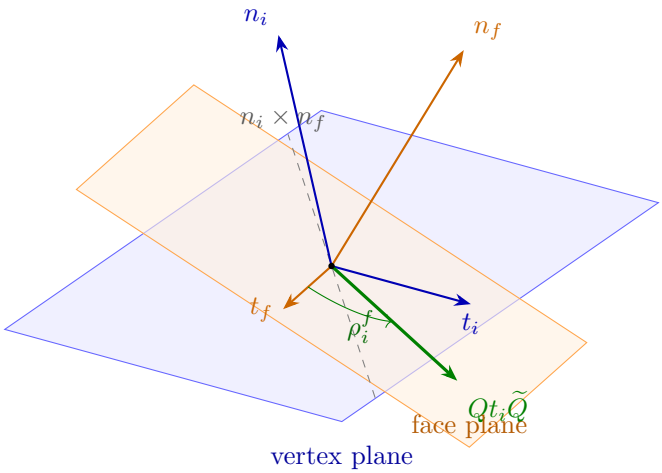


FIGURE 5. The discrete transport from vertex i into face f. The smallest rotation Q, about the axis $n_i \times n_f$ (dashed), takes the vertex plane (blue) onto the face plane (orange); the frame vector t_i becomes $Qt_i\widetilde{Q}$ (green, written as a rotor as in Part 2), which makes the angle ρ_i^f with the face frame vector t_f. The matrix R_i^f is the rotation by ρ_i^f.

and $v = n_i \times n_f = (1/\sqrt{2}, 0, 0)$*: Q is the rotation by* $45°$ *about the x axis, and it takes* $t_i = (0,1,0)$ *to* $(0,1,1)/\sqrt{2} = b_f$ *and* b_i *to itself. So the transported* t_i *is the second face frame vector, and*

$$R_i^f = T_f^T Q T_i = \begin{pmatrix} t_f \cdot Qt_i & t_f \cdot Qb_i \\ b_f \cdot Qt_i & b_f \cdot Qb_i \end{pmatrix} = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix},$$

the rotation by $\rho_i^f = 90°$*. The program* `ga_fem` *prints these numbers (Section 17).*

A rotation of the plane is multiplication by a unit complex number, $R_i^f \leftrightarrow e^{\mathrm{i}\rho_i^f}$: the angle ρ_i^f is the discrete connection between the frame of vertex i and that of face f. The angle has two contributions, the turning of the tangent plane, from Q, and the arbitrary choice of the two frames, which is why only combinations in which the frame choices cancel have a geometric meaning.

Transport between two vertices. To bring a vector from vertex j to vertex i through a face f that contains both, go from j into the face and from the face back to i:

$$R_{j\to i}^f = (R_i^f)^{-1} R_j^f = (R_i^f)^T R_j^f. \tag{3.6}$$

Here the choice of the face frame cancels: replacing T_f by $T_f S$ for a rotation S of the plane changes R_i^f to $S^T R_i^f$ and R_j^f to $S^T R_j^f$, and leaves $(R_i^f)^T R_j^f$ unchanged. The choices of the vertex frames do not cancel, and

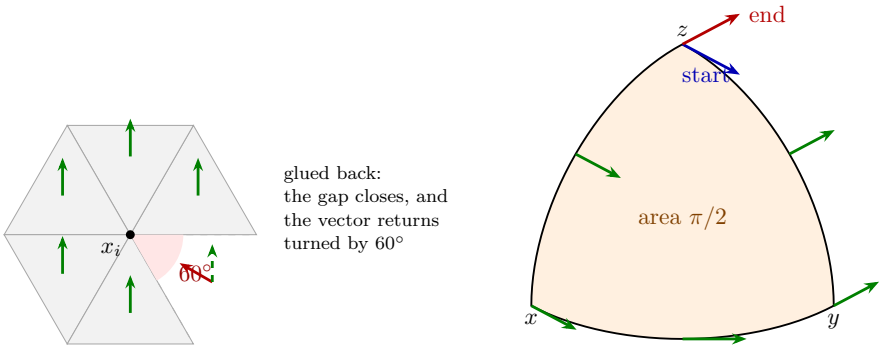


FIGURE 6. Holonomy. Left: the one-ring of a vertex of five equilateral triangles, unfolded into the plane; its angles add up to 300°, and a vector transported around the vertex (constant in the unfolded plane) comes back turned by the angle defect 60° when the gap is closed. Right: on the unit sphere, a vector carried around the octant bounded by three quarter great circles comes back turned by 90°, the area $\pi/2$ of the octant.

should not: they define the coordinates of the unknowns. This product is the matrix that will appear in the stiffness matrix. The first version of the code, `AssembleStiffnessElementEmbedded`, uses instead the smallest rotation from n_j to n_i directly, $\texttt{Rvv}(v_j, v_i) = T_i^T Q_{j\to i} T_j$, which is the same to first order in the angles between the normals; the version used, `AssembleStiffnessElementEmbedded2`, goes through the face as in (3.6), which is what the finite element derivation of Section 6 produces.

Remark 3.4 (Curvature). *Going around a vertex from face to face, the transports (3.6) through consecutive faces compose to a rotation that is not the identity in general. For the intrinsic transport of the polyhedral surface (unfolding the faces one after another) its angle is the* angle defect $2\pi - \sum_f \alpha_f$ *of the vertex, the discrete Gaussian curvature; for the transport through vertex normals used here, it approximates the angle defect on fine meshes (Section 14). This is the path dependence of parallel transport mentioned in the introduction, and the reason why the transported field must be defined globally by an equation, rather than by composing transports along one particular path.*

Worked example 3.5. *Five equilateral triangles around a vertex (Figure 6, left) form a cone with total angle* $5 \cdot 60° = 300°$*, a pentagonal pyramid. Cutting it along an edge and laying it flat leaves a gap of* 60°*. A vector carried around the vertex keeps its direction in the flat layout, and when the gap is glued shut it meets its starting copy turned by* $60° = 2\pi - 300°$*, the angle defect. On the unit sphere (right), start at the north pole with a vector, go down a meridian to the equator, along the equator by a quarter turn, and*

up another meridian back to the pole. Along each great circle the vector keeps its angle with the direction of motion, and it returns turned by $90°$*, which is the area* $4\pi/8 = \pi/2$ *of the octant: on a sphere of curvature* $K = 1$*, holonomy is enclosed area.*

4. The Connection Laplacian and the Vector Heat Equation

4.1. The Dirichlet energy and the connection Laplacian

How much does a field turn? The natural measure is the *Dirichlet energy*

$$E(U) = \frac{1}{2}\int_\Omega |\nabla U|^2\, dA, \qquad |\nabla U|^2 = \nabla U : \nabla U = \sum_{k=1}^{2} |\nabla_{e_k} U|^2, \tag{4.1}$$

where e_1, e_2 is any orthonormal basis of the tangent plane and $A : B = \operatorname{tr}(A^T B)$ is the inner product of 2×2 matrices. $E(U) = 0$ exactly when $\nabla U = 0$ everywhere, that is, when U is parallel along every curve; on a curved surface such fields do not exist, and the fields of smallest energy are the "most parallel" ones.

The operator associated with this energy is found, as always, by differentiating it. For a variation $U + \epsilon V$, the derivative of E at $\epsilon = 0$ is $\int \nabla U : \nabla V\, dA$, and we want to write it as $\int (\cdots) \cdot V\, dA$, with no derivative on V. The tool is Green's identity for tangent fields.

Proposition 4.1 (Green's identity). *For tangent fields* U *and* V *on a surface* Ω *with boundary* $\partial\Omega$ *and outward unit conormal* ν*,*

$$\int_\Omega \nabla U : \nabla V\, dA = \int_\Omega (\nabla^*\nabla U) \cdot V\, dA + \oint_{\partial\Omega} (\nabla_\nu U) \cdot V\, ds, \tag{4.2}$$

where the connection Laplacian *is* $\nabla^*\nabla U = -\sum_k \left(\nabla_{e_k}\nabla_{e_k} U - \nabla_{\nabla_{e_k} e_k} U\right)$.

Proof. Consider the tangent field W defined by $W \cdot X = (\nabla_X U) \cdot V$ for every tangent X. Its divergence, computed with the connection, is $\operatorname{div} W = \sum_k (\nabla_{e_k} W) \cdot e_k$, and by the metric compatibility (3.2) applied twice,

$$\begin{aligned} \operatorname{div} W &= \sum_k \Big(\left(\nabla_{e_k}\nabla_{e_k} U - \nabla_{\nabla_{e_k} e_k} U\right) \cdot V + \nabla_{e_k} U \cdot \nabla_{e_k} V \Big) \\ &= -(\nabla^*\nabla U) \cdot V + \nabla U : \nabla V. \end{aligned}$$

Integrating over Ω and using the divergence theorem, $\int_\Omega \operatorname{div} W\, dA = \oint_{\partial\Omega} W \cdot \nu\, ds = \oint (\nabla_\nu U) \cdot V\, ds$, gives (4.2). □

With V vanishing on the boundary, (4.2) reads $\int \nabla U : \nabla V = \int (\nabla^*\nabla U) \cdot V$: $\nabla^*\nabla$ is the *adjoint* of ∇, hence the notation. Taking $V = U$ shows that it is *positive*: $\int (\nabla^*\nabla U) \cdot U\, dA = \int |\nabla U|^2\, dA \geq 0$. Its negative, $\Delta U = -\nabla^*\nabla U$, is the *rough* or *Bochner* Laplacian of tangent fields; on scalar functions the same construction gives the familiar Laplace–Beltrami operator. The sign matters: the energy decreases along $-\nabla^*\nabla U$, which is the direction in which the heat equation must move.

4.2. The vector heat equation

The heat equation for tangent fields lets a field flow downhill in the Dirichlet energy:

$$\frac{\partial U}{\partial t} = -\nabla^* \nabla U = \Delta U. \tag{4.3}$$

By Green's identity, $\frac{d}{dt}\frac{1}{2}\int |U|^2 = -\int |\nabla U|^2 \leq 0$ (with $U = 0$ or $\nabla_\nu U = 0$ on the boundary): the flow smooths the field and decays it. Starting from a vector concentrated at a source point s, Sharp, Soliman and Crane [9] show that for short times the solution at a point x points in the direction of the source vector transported along the shortest geodesic from s to x, and that only its magnitude depends on the distance. Normalizing the solution at every point therefore gives the parallel transport of the source vector. (Their method also diffuses the magnitude separately, with a scalar heat equation, to recover a field of prescribed length; here we need only the direction.)

4.3. Backward Euler

We take one time step of length Δt with the *backward* Euler method, which evaluates the right-hand side of (4.3) at the new time: with U^0 at time 0 and U at time Δt,

$$\frac{U - U^0}{\Delta t} = -\nabla^* \nabla U \qquad \Longleftrightarrow \qquad U + \Delta t\, \nabla^* \nabla U = U^0. \tag{4.4}$$

The operator $I + \Delta t\, \nabla^* \nabla$ is positive definite, because $\nabla^* \nabla$ is positive, so the step is always solvable and stable for any Δt, which is why the implicit method is used: a single large step can be taken. The vector heat method takes exactly one step, with Δt of the order of the square of the mesh spacing h, the time it takes heat to cross a few elements; the code uses $\Delta t = \bar{h}^2$ with $\bar{h}$ the mean edge length (`computeEdgeLengths`).

The boundary condition used by the code. The source can be imposed in two ways. Sharp et al. put it in the initial condition, $U^0 = \delta_s U_s$, a vector concentrated at the source vertex. The code instead prescribes the value at the source vertex and starts from zero everywhere else: it solves

$$U + \Delta t\, \nabla^* \nabla U = 0 \quad \text{away from } s, \qquad U(s) = U_s, \tag{4.5}$$

a *Dirichlet* condition at one point. Both produce a field that decays away from the source and whose direction is transported along the paths of steepest heat flow; the direction, which is all we keep, is the same to the accuracy of the method. Equation (4.5) is a *screened* Poisson equation for the connection Laplacian: the term U makes the influence of the source decay over a distance of order $\sqrt{\Delta t} \approx \bar{h}$.

5. The Weak Form

The equation (4.4) involves second derivatives of U, through $\nabla^* \nabla$, and a piecewise linear field on a mesh has no second derivatives: its first derivatives

jump across edges. The weak form removes the second derivatives by moving one of them onto a test field.

Test fields. Multiply (4.4) by an arbitrary tangent field V, a *test field*, take the inner product, and integrate:

$$\int_\Omega U \cdot V \, dA + \Delta t \int_\Omega (\nabla^* \nabla U) \cdot V \, dA = \int_\Omega U^0 \cdot V \, dA.$$

If this holds for every test field V, the equation holds at every point (take V concentrated near any point). Conversely it suffices to ask it for every V in a large enough family; this is the idea that the finite element method will exploit with a finite family.

Integration by parts. By Green's identity (4.2), the second term equals $\int \nabla U : \nabla V$ minus a boundary term. Our surfaces are closed, or we choose test fields that vanish where U is prescribed, so the boundary term disappears, and the *weak form* of the backward Euler step is: find U such that

$$\boxed{\int_\Omega U \cdot V \, dA + \Delta t \int_\Omega \nabla U : \nabla V \, dA = \int_\Omega U^0 \cdot V \, dA} \tag{5.1}$$

for every test field V. Only first derivatives appear, of U and of V symmetrically. The equation now makes sense for fields whose derivatives are merely square integrable, including continuous, piecewise linear ones, which is the meaning of "weak". And it is the condition that the energy

$$J(U) = \frac{1}{2} \int_\Omega |U|^2 \, dA + \frac{\Delta t}{2} \int_\Omega |\nabla U|^2 \, dA - \int_\Omega U^0 \cdot U \, dA \tag{5.2}$$

be stationary, since its derivative in the direction V is the left-hand side of (5.1) minus its right-hand side. Being a sum of squares plus a linear term, J is strictly convex, and its unique minimizer is the solution: the backward Euler step looks for the field closest to U^0 that is also as parallel as possible, with Δt trading one against the other.

6. The Galerkin Method: Piecewise Linear Tangent Fields

The Galerkin method replaces "every test field" by a finite family, and the unknown field by a combination of the same family. We need, then, a finite dimensional space of tangent fields on the mesh. For scalar functions the standard choice is the piecewise linear functions; this section builds their analogue for tangent fields, and shows that the connection is what makes it possible.

6.1. The scalar hat functions

For each vertex i, the *hat function* ϕ_i is the continuous function on the mesh that is linear on every face, equals 1 at x_i and 0 at every other vertex. It is nonzero only on the faces around i (its *support*), it is a tent over the one-ring of i, and on each face f the three hat functions of its vertices add up to 1, since $\phi_{f_1} + \phi_{f_2} + \phi_{f_3}$ is linear and equals 1 at the three vertices. The scalar

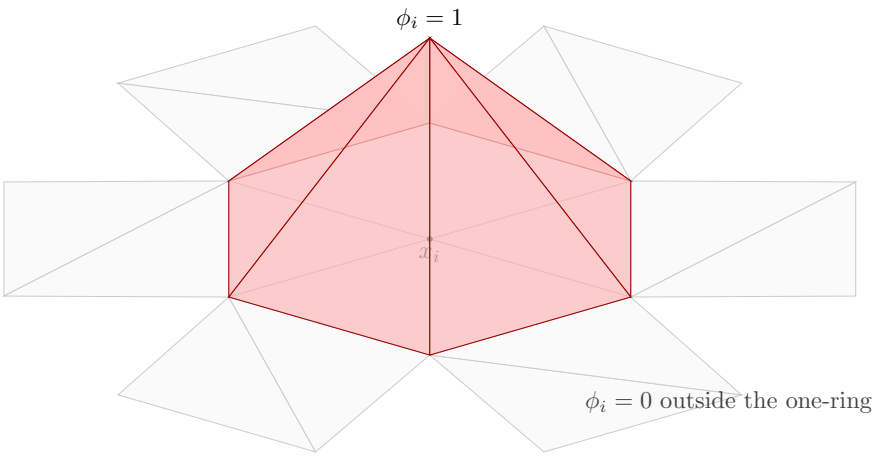


FIGURE 7. The hat function ϕ_i of a vertex: linear on every face, 1 at x_i, 0 at all other vertices, and therefore zero outside the one-ring of x_i.

piecewise linear functions are the combinations $\sum_i c_i \phi_i$, and c_i is the value at vertex i.

6.2. Why the vector case needs a connection

The obvious vector analogue would be $U = \sum_i \phi_i \, (T_i u_i)$: interpolate the three vertex vectors of each face with the hat functions. But $T_i u_i$ lies in the tangent plane of vertex i, and the three vertices of a face have three different tangent planes: the interpolated vector would not be tangent to the face, and its derivative would contain the bending of the surface rather than the turning of the field. Before interpolating, the three vectors must be brought into one plane, the plane of the face, and the rule that brings a vector from one tangent plane into another is parallel transport. So, on each face f, we interpolate the *transported* vertex vectors:

$$U\big|_f = \sum_{i \in f} \phi_i \; T_f R_i^f u_i, \qquad \text{in face coordinates:} \quad T_f^T U\big|_f = \sum_{i \in f} \phi_i \, R_i^f u_i. \tag{6.1}$$

Figure 8 shows the difference. This defines a tangent field on every face, linear in the unknowns u_i. The space of such fields is spanned by the $2n$ *vector hat functions*

$$\Psi_{i,k}\big|_f = \phi_i \; T_f R_i^f \varepsilon_k, \qquad k = 1, 2, \tag{6.2}$$

where $\varepsilon_1 = (1, 0)$ and $\varepsilon_2 = (0, 1)$: the frame vector k of vertex i, transported into each incident face, and faded out by the scalar hat. Then $U = \sum_{i,k} u_{i,k} \Psi_{i,k}$, with the unknowns (2.5) as coefficients. At vertex i itself, $\phi_i = 1$ and the other hats vanish, so the field takes, in each incident face, the transported value $T_f R_i^f u_i$: the vertex vector seen from that face.

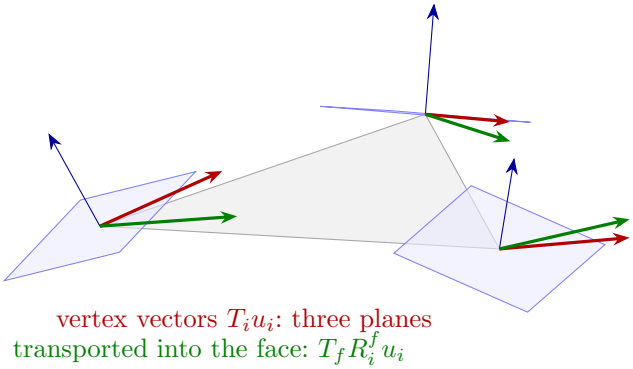


FIGURE 8. Why vectors must be transported before they are interpolated. The vertex vectors $T_i u_i$ (red) lie in three different vertex planes (blue), none of which is the plane of the face; transported into the face, $T_f R_i^f u_i$ (green), they lie in one plane and can be combined.

6.3. The covariant derivative of a basis function

Inside a face, the surface is a plane and the face frame T_f is constant, so the connection form vanishes, and by (3.3) the covariant derivative is the ordinary derivative of the face coordinates. In (6.1) the transported vectors $R_i^f u_i$ are constant on the face; only the hat functions vary. Hence, for a tangent direction X of the face,

$$T_f^T \nabla_X U\big|_f = \sum_{i\in f} (X\cdot\nabla\phi_i)\, R_i^f u_i, \qquad T_f^T \nabla U\big|_f = \sum_{i\in f} (R_i^f u_i)\,(g_i^f)^T, \tag{6.3}$$

where $g_i^f = T_f^T \nabla\phi_i \in \mathbb{R}^2$ is the gradient of the hat function in the coordinates of the face, a constant vector on the face, computed in Section 7. The total covariant derivative is a sum of rank-one 2×2 matrices: the transported vertex vector times the gradient of its hat. In tensor notation, $\nabla\Psi_{i,k} = (R_i^f \varepsilon_k)\otimes g_i^f$.

6.4. The discrete equations

Now take $U = \sum_{j,l} u_{j,l}\Psi_{j,l}$ and, as test fields, every basis function $V = \Psi_{i,k}$ (the *Galerkin* choice: test and trial spaces are the same). The weak form (5.1) becomes $2n$ linear equations,

$$\sum_{j,l}\Big(\int_\Omega \Psi_{j,l}\cdot\Psi_{i,k}\,dA + \Delta t\int_\Omega \nabla\Psi_{j,l} : \nabla\Psi_{i,k}\,dA\Big)\,u_{j,l} = \int_\Omega U^0\cdot\Psi_{i,k}\,dA, \tag{6.4}$$

which, in matrix form, read

$$(\mathrm{M} + \Delta t\,\mathrm{L})\,\mathrm{u} = \mathrm{M}\,\mathrm{u}^0, \tag{6.5}$$

with the *mass matrix* M and the *stiffness matrix* L, both $2n \times 2n$ and both made of 2×2 blocks, one per pair of vertices (i, j):

$$\mathrm{M}_{ij} = \Big[\int_\Omega \Psi_{j,l} \cdot \Psi_{i,k} \, dA \Big]_{k,l}, \qquad \mathrm{L}_{ij} = \Big[\int_\Omega \nabla\Psi_{j,l} : \nabla\Psi_{i,k} \, dA \Big]_{k,l}. \tag{6.6}$$

(For the right-hand side we have written $U^0 = \sum_j \Psi_{j,l} \, u^0_{j,l}$, its interpolant.) Each integral is a sum over faces, and a face contributes only if it contains both i and j, because the hats vanish elsewhere. So the matrices are sparse, with a nonzero block for each vertex and for each pair of vertices joined by an edge, and they can be computed face by face, which is the next step.

The stiffness block of a face. On a face f containing i and j, by (6.3) and the identity $(ab^T) : (cd^T) = (a \cdot c)(b \cdot d)$,

$$\nabla\Psi_{j,l} : \nabla\Psi_{i,k} = \big((R^f_j \varepsilon_l)(g^f_j)^T\big) : \big((R^f_i \varepsilon_k)(g^f_i)^T\big) = (g^f_i \cdot g^f_j)\,(R^f_i \varepsilon_k) \cdot (R^f_j \varepsilon_l),$$

and $(R^f_i \varepsilon_k) \cdot (R^f_j \varepsilon_l) = \varepsilon_k^T (R^f_i)^T R^f_j \varepsilon_l$ is the entry (k, l) of $(R^f_i)^T R^f_j$. The integrand is constant on the face, so its integral is the area times the constant:

$$\boxed{\mathrm{L}^f_{ij} = A_f \, (g^f_i \cdot g^f_j)\, (R^f_i)^T R^f_j} \tag{6.7}$$

This is the heart of the method, and it is worth reading slowly. The scalar factor $A_f \, g^f_i \cdot g^f_j$ is the entry (i, j) of the stiffness matrix of the ordinary scalar Laplacian; the 2×2 factor $(R^f_i)^T R^f_j$ is, by (3.6), the parallel transport from vertex j to vertex i through the face. The connection Laplacian is the scalar Laplacian in which every coupling between two vertices first transports the vector of one into the frame of the other. For $i = j$ the transport is the identity, $(R^f_i)^T R^f_i = I$, and the diagonal block is the scalar diagonal entry times I. On a flat mesh with a common frame, every $R^f_i = I$, and the connection Laplacian is two copies of the scalar Laplacian, one per coordinate.

The block for (j, i) is $A_f (g^f_j \cdot g^f_i)(R^f_j)^T R^f_i = (\mathrm{L}^f_{ij})^T$, so the matrix is symmetric. It is also positive semi-definite, because $\mathrm{u}^T \mathrm{L} \mathrm{u} = \int |\nabla U|^2 \, dA \geq 0$ for the field U of coefficients u.

7. Shape Functions and Their Gradients on a Triangle in Space

The element matrices need the constant gradients g^f_i of the three hat functions on each face. The standard way to compute them is to do the work once, on a fixed *reference triangle*, and to carry the result to every face of the mesh with an affine map [8].

7.1. The reference triangle

The reference triangle $\hat{K}$ has the vertices $\hat{P}_1 = (0, 0)$, $\hat{P}_2 = (1, 0)$, $\hat{P}_3 = (0, 1)$ in the plane of the coordinates $\xi = (\xi_1, \xi_2)$:

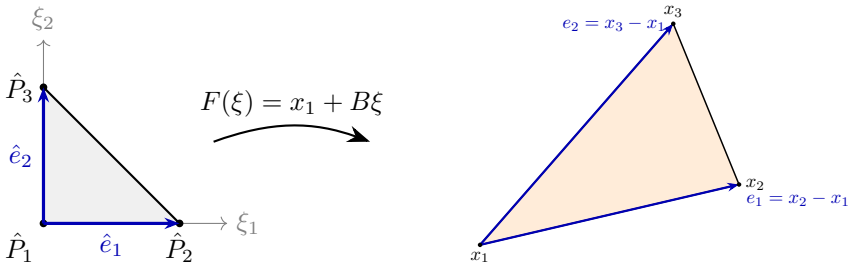
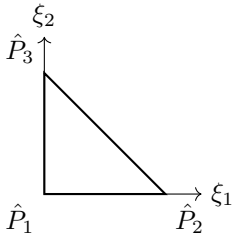


FIGURE 9. The affine map from the reference triangle to a face in space. The columns of B are the images of the reference edges.



Its three linear *shape functions*, each equal to 1 at one vertex and 0 at the other two, are

$$\hat{\phi}_1(\xi) = 1 - \xi_1 - \xi_2, \qquad \hat{\phi}_2(\xi) = \xi_1, \qquad \hat{\phi}_3(\xi) = \xi_2, \tag{7.1}$$

as one checks at the three vertices: $\hat{\phi}_a(\hat{P}_b) = \delta_{ab}$. Their gradients are constant:

$$\hat{\nabla}\hat{\phi}_1 = (-1, -1), \qquad \hat{\nabla}\hat{\phi}_2 = (1, 0), \qquad \hat{\nabla}\hat{\phi}_3 = (0, 1). \tag{7.2}$$

7.2. The map to a face in space

The affine map that takes the reference triangle onto a face f with vertices x_1, x_2, x_3 (local numbering, in the order of the face) is

$$F(\xi) = x_1 + B\,\xi, \qquad B = \begin{bmatrix} x_2 - x_1 & x_3 - x_1 \end{bmatrix} \in \mathbb{R}^{3\times 2}. \tag{7.3}$$

It takes $\hat{P}_1, \hat{P}_2, \hat{P}_3$ to x_1, x_2, x_3, and the edges $\hat{e}_1, \hat{e}_2$ of the reference triangle to the edges $e_1 = x_2 - x_1$ and $e_2 = x_3 - x_1$ of the face, the columns of B (Figure 9).

The hat functions on the face are the reference shape functions composed with the inverse of the map: $\phi_a = \hat{\phi}_a \circ F^{-1}$. In the plane we would invert B, but here B is 3×2: it maps the plane of ξ onto the plane of the face, which is a plane in space, and has no inverse. What we need is the gradient of ϕ_a, a vector in the plane of the face, and the chain rule determines it.

Proposition 7.1. *On the face,* $\nabla\phi_a = B(B^TB)^{-1}\,\hat{\nabla}\hat{\phi}_a$, *and this vector lies in the plane of the face.*

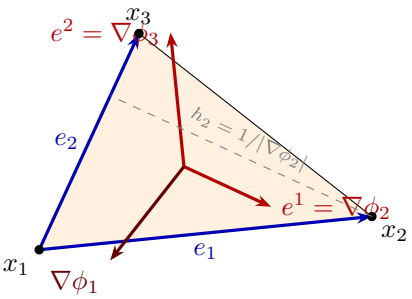


FIGURE 10. The gradients of the hat functions of a face. $\nabla\phi_2$ is orthogonal to the edge opposite to x_2, points towards x_2, and has length $1/h_2$, the reciprocal of the height; the gradients $\nabla\phi_2 = e^1$ and $\nabla\phi_3 = e^2$ form the *reciprocal frame* of the edges e_1, e_2 ($e^k \cdot e_m = \delta_{km}$), and the three gradients add up to zero.

Proof. Let $g = \nabla\phi_a$ be the gradient, a vector in the plane of the face. Its defining property is that, along any direction X of the face, the derivative of ϕ_a is $g \cdot X$. The edges $B\hat{e}_1 = x_2 - x_1$ and $B\hat{e}_2 = x_3 - x_1$ are two such directions, and along them $\phi_a(F(\xi)) = \hat{\phi}_a(\xi)$ changes at the rates $\partial\hat{\phi}_a/\partial\xi_m$: so $B^T g = \hat{\nabla}\hat{\phi}_a$, two equations. Since g lies in the plane spanned by the columns of B, $g = Bc$ for some $c \in \mathbb{R}^2$, and then $B^T Bc = \hat{\nabla}\hat{\phi}_a$, so $c = (B^T B)^{-1}\hat{\nabla}\hat{\phi}_a$, and $g = B(B^T B)^{-1}\hat{\nabla}\hat{\phi}_a$. The matrix $B^T B$ is the Gram matrix of the two edges, invertible for a non-degenerate triangle. □

The matrix $B(B^T B)^{-1}$ is the transpose of the Moore–Penrose pseudo-inverse $B^+ = (B^T B)^{-1} B^T$ of B, and for a triangle in the plane, where B is square, it reduces to B^{-T}, the familiar formula of the planar finite element method. In the code, `Binv` $= ((B^T B)^{-1} B^T)^T = B(B^T B)^{-1}$, and the three gradients are its columns combined as in (7.2):

$$\nabla\phi_1 = -C_{:,1} - C_{:,2}, \qquad \nabla\phi_2 = C_{:,1}, \qquad \nabla\phi_3 = C_{:,2}, \qquad C = \texttt{Binv}, \quad (7.4)$$

which is `gradN[0]`, `gradN[1]`, `gradN[2]` in `AssembleStiffnessElement-Embedded2` (and in `Gradient`). Note that the three gradients add up to zero, as they must, since the three hat functions add up to 1. Finally, the coordinates of the gradient in the face frame are

$$g_a^f = T_f^T \nabla\phi_a \in \mathbb{R}^2, \quad (7.5)$$

`grad_i = Tf.transpose() * gradN[i]`. Since $\nabla\phi_a$ lies in the face plane and the frame is orthonormal, $g_a^f \cdot g_b^f = \nabla\phi_a \cdot \nabla\phi_b$: the projection does not change the stiffness, it only expresses the gradients in the same coordinates as the transported vectors.

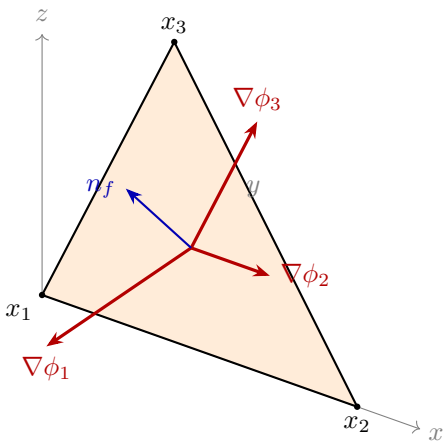


FIGURE 11. The triangle of Example 7.2, its normal $n_f = (0,-1,1)/\sqrt{2}$, and the gradients of its three hat functions, drawn from the centroid. All three lie in the plane of the face.

Worked example 7.2. *Take the triangle* $x_1 = (0,0,0)$, $x_2 = (2,0,0)$, $x_3 = (0,1,1)$ *of Figure 11. Its edges are* $e_1 = (2,0,0)$ *and* $e_2 = (0,1,1)$, *so*

$$B = \begin{pmatrix} 2 & 0 \\ 0 & 1 \\ 0 & 1 \end{pmatrix}, \quad B^T B = \begin{pmatrix} 4 & 0 \\ 0 & 2 \end{pmatrix}, \quad B(B^T B)^{-1} = \begin{pmatrix} 1/2 & 0 \\ 0 & 1/2 \\ 0 & 1/2 \end{pmatrix}.$$

The edges are orthogonal, so $B^T B$ *is diagonal and the reciprocal frame is just* $e_k/|e_k|^2$. *By (7.4),*

$$\nabla\phi_1 = \left(-\tfrac{1}{2}, -\tfrac{1}{2}, -\tfrac{1}{2}\right), \qquad \nabla\phi_2 = \left(\tfrac{1}{2}, 0, 0\right), \qquad \nabla\phi_3 = \left(0, \tfrac{1}{2}, \tfrac{1}{2}\right),$$

which add up to zero. Check $\nabla\phi_2$: *along* e_1, ϕ_2 *goes from* 0 *to* 1, *and* $\nabla\phi_2 \cdot e_1 = 1$; *along* e_2 *it stays* 0, *and* $\nabla\phi_2 \cdot e_2 = 0$. *The area is* $A_f = \frac{1}{2}|e_1 \times e_2| = \frac{1}{2}|(0,-2,2)| = \sqrt{2}$, *and the scalar stiffness* $A_f\, \nabla\phi_a \cdot \nabla\phi_b$ *of the face is*

$$\sqrt{2}\begin{pmatrix} 3/4 & -1/4 & -1/2 \\ -1/4 & 1/4 & 0 \\ -1/2 & 0 & 1/2 \end{pmatrix} = \begin{pmatrix} 1.0607 & -0.3536 & -0.7071 \\ -0.3536 & 0.3536 & 0 \\ -0.7071 & 0 & 0.7071 \end{pmatrix},$$

whose rows add up to zero, since constants have zero gradient. The entry $(2,3)$ *vanishes because the angle at* x_1 *is a right angle, in agreement with the cotangent formula of the next section; the entry* $(1,2)$ *is* $-\frac{1}{2}\cot\theta_3$, *where* $\cos\theta_3 = \frac{(x_1-x_3)\cdot(x_2-x_3)}{|x_1-x_3|\,|x_2-x_3|} = \frac{2}{\sqrt{2}\sqrt{6}} = \frac{1}{\sqrt{3}}$, *so* $\cot\theta_3 = \frac{1}{\sqrt{2}}$ *and* $-\frac{1}{2}\cot\theta_3 = -0.3536$.

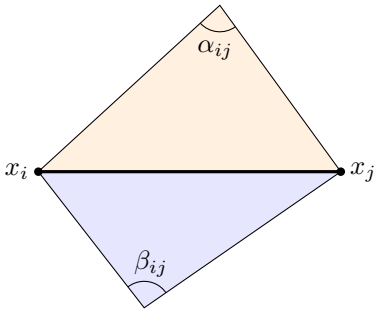


FIGURE 12. The two angles opposite to an edge ij in its two faces. The scalar part of the stiffness block of the edge is the cotangent weight; the connection Laplacian multiplies it by the transport from j to i.

7.3. A check: the cotangent formula

The scalar factor of (6.7) has a classical closed form, which connects the finite element computation with the cotangent Laplacian of geometry processing (`CalcCotangentWeights` in `Laplacian.cpp`).

Proposition 7.3. *For two distinct vertices a, b of a face with area A_f, $A_f\,\nabla\phi_a \cdot \nabla\phi_b = -\frac{1}{2}\cot\theta_c$, where θ_c is the angle at the third vertex c; and $A_f\,|\nabla\phi_a|^2 = \frac{1}{2}(\cot\theta_b + \cot\theta_c)$.*

Proof. $\nabla\phi_a$ is orthogonal to the edge opposite to a (along which ϕ_a is constant, zero), points towards a, and has length $1/h_a$, where $h_a = 2A_f/\ell_a$ is the height from a and ℓ_a the length of the opposite edge. So $\nabla\phi_a = \frac{\ell_a}{2A_f} m_a$ with m_a the unit inward normal of the opposite edge, and $\nabla\phi_a \cdot \nabla\phi_b = \frac{\ell_a \ell_b}{4A_f^2}\, m_a \cdot m_b$. The inward normals of two edges meeting at c make the angle $\pi - \theta_c$, so $m_a \cdot m_b = -\cos\theta_c$, and with $2A_f = \ell_a \ell_b \sin\theta_c$ we get $A_f\,\nabla\phi_a \cdot \nabla\phi_b = -\frac{\ell_a \ell_b \cos\theta_c}{4A_f} = -\frac{1}{2}\cot\theta_c$. The diagonal follows from $\nabla\phi_a = -\nabla\phi_b - \nabla\phi_c$. □

Summing over the two faces of an edge ij, the off-diagonal stiffness is $-\frac{1}{2}(\cot\alpha_{ij} + \cot\beta_{ij})$, the cotangent weight of the edge. (Figure 12.) The connection Laplacian (6.7) is therefore the cotangent Laplacian with a 2×2 transport on every edge, as in the discrete connection Laplacians of geometry processing [4, 9]; the finite element derivation explains where both factors come from.

8. The Element Matrices

8.1. The element stiffness matrix

On one face, the hat functions of its three vertices are the only nonzero ones, so the face contributes to the 3×3 blocks of L indexed by its vertices, a 6×6 *element matrix* in the local numbering $a, b \in \{1, 2, 3\}$:

$$\mathrm{L}^f = \begin{pmatrix} \mathrm{L}^f_{11} & \mathrm{L}^f_{12} & \mathrm{L}^f_{13} \\ \mathrm{L}^f_{21} & \mathrm{L}^f_{22} & \mathrm{L}^f_{23} \\ \mathrm{L}^f_{31} & \mathrm{L}^f_{32} & \mathrm{L}^f_{33} \end{pmatrix}, \qquad \mathrm{L}^f_{ab} = A_f \, (g^f_a \cdot g^f_b) \, (R^f_a)^T R^f_b \in \mathbb{R}^{2\times 2}. \quad (8.1)$$

This is `AssembleStiffnessElementEmbedded2`: it computes B, `Binv`, A_f (`faceArea`), the three gradients, their face coordinates g^f_a and the transports R^f_a (`Rvf`), and fills the blocks with `elementMatrix.block<2,2>(a*2, b*2)`. Following the symmetry $\mathrm{L}^f_{ba} = (\mathrm{L}^f_{ab})^T$, it loops over $a \geq b$ only and writes both blocks; on the diagonal it writes $A_f |g^f_a|^2 I_2$.

Exactness of the integration. The integrand of (6.7) is constant on the face, so "area times integrand" is the exact integral, with no quadrature error. For higher order elements the gradients would vary and a quadrature rule would be needed; with linear elements it is not.

8.2. The element mass matrix and its lumping

The mass matrix needs $\int_f \Psi_{b,l} \cdot \Psi_{a,k} \, dA = \left(\int_f \phi_a \phi_b \, dA \right) \left((R^f_a)^T R^f_b \right)_{kl}$, by the same computation as for the stiffness but without derivatives. The integrals of products of linear hats over a triangle are $\int_f \phi_a \phi_b \, dA = \frac{A_f}{12}(1 + \delta_{ab})$, so the *consistent* element mass matrix is

$$\mathrm{M}^f_{ab} = \frac{A_f}{12}(1 + \delta_{ab}) \, (R^f_a)^T R^f_b. \quad (8.2)$$

The code uses the simpler *lumped* mass matrix, which moves all the mass of each row to the diagonal. Since the three hats add up to 1 on the face, the row sums of the scalar factor are $\sum_b \int_f \phi_a \phi_b = \int_f \phi_a = \frac{A_f}{3}$, and the lumped element mass matrix is

$$\mathrm{M}^f_{aa} = \frac{A_f}{3} \, I_2, \qquad \mathrm{M}^f_{ab} = 0 \quad (a \neq b), \quad (8.3)$$

where the transport has disappeared because $(R^f_a)^T R^f_a = I$. Assembled, the mass of vertex i is one third of the area of its incident faces, the same for both coordinates. This is `AssembleDiagonalMassMatrix`. Lumping changes the discrete solution by an amount of the order of the discretization error, keeps M diagonal and positive, and makes M^{-1} trivial, which the code uses next.

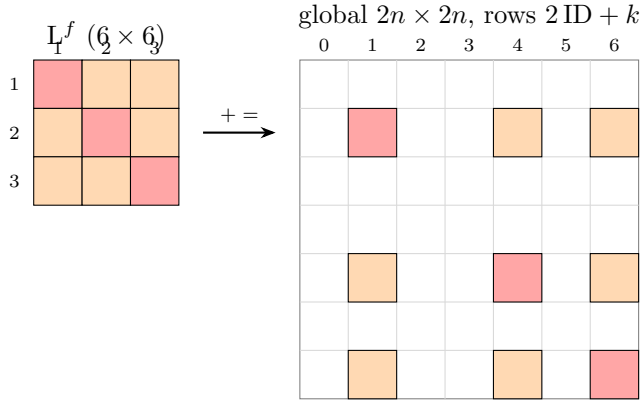


FIGURE 13. Assembly. The 6×6 element matrix of a face, made of 3×3 blocks of size 2×2 (diagonal blocks red), is added into the global matrix at the rows and columns of its vertices, here 1, 4 and 6.

9. Assembly

From local to global indices. The global matrices are sums of the element matrices, each placed at the rows and columns of the vertices of its face:

$$\mathrm{L} = \sum_f P_f^T \, \mathrm{L}^f \, P_f, \qquad \mathrm{M} = \sum_f P_f^T \, \mathrm{M}^f \, P_f, \tag{9.1}$$

where P_f is the $6 \times 2n$ matrix that extracts, from the global vector u, the six coordinates of the vertices of f in local order. In code, nobody forms P_f: the local block (a, b) of L^f, with entries (k, l), is added to the global entry

$$\big(2\,\mathrm{ID}(f_a) + k,\ 2\,\mathrm{ID}(f_b) + l\big), \quad k, l \in \{0, 1\}, \tag{9.2}$$

with the zero-based vertex indices ID of the mesh: `(*A)(v[i]->ID * 2 + row, v[j]->ID * 2 + col) += wij(row, col)`. The function `AssembleMatrix2` loops over the faces, computes L^f, and adds $\Delta t\, \mathrm{L}^f$ to the global matrix, so that it returns $\Delta t\, \mathrm{L}$; `AssembleDiagonalMassMatrix` does the same with (8.3). Each interior edge gets contributions from its two faces, and each vertex from all its incident faces: the sums (9.1) are exactly the sums over faces of the integrals in (6.6). Figure 13 shows one face being added.

Structure. The assembled L has a 2×2 block for each vertex (the diagonal) and for each edge, in both orders, with $\mathrm{L}_{ji} = \mathrm{L}_{ij}^T$. It is symmetric and positive semi-definite. It is singular on closed surfaces only if a parallel field exists, which on a curved closed surface is never the case, so in practice $\mathrm{M} + \Delta t\, \mathrm{L}$ is positive definite even before the boundary condition.

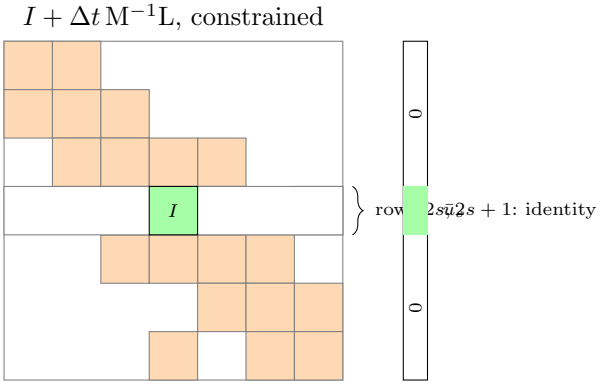


FIGURE 14. The Dirichlet condition by row replacement: the two rows of the source vertex s become rows of the identity, and the right-hand side holds the prescribed vector $\bar{u}_s$ there and zero elsewhere.

10. The Linear System and Its Solution

10.1. The system the code solves

The discrete backward Euler step is (6.5), $(\mathrm{M} + \Delta t\, \mathrm{L})\, \mathrm{u} = \mathrm{M}\, \mathrm{u}^0$. Multiplying by the diagonal M^{-1} on the left, which does not change the solution,

$$(I + \Delta t\, \mathrm{M}^{-1}\mathrm{L})\, \mathrm{u} = \mathrm{u}^0. \tag{10.1}$$

This is the form the code builds, in place: `IplusMinvTimesA` divides each row of $\Delta t\, \mathrm{L}$ by the diagonal mass of that row and adds 1 to the diagonal. The scaled matrix is no longer symmetric (its rows are scaled by different masses), which is one reason why the code factors it with a general sparse LU. The symmetric form (6.5) could be factored with a Cholesky factorization instead, at half the cost; the two have the same solution.

10.2. The Dirichlet condition

The source vector is prescribed at one vertex s: $u_s = \bar{u}_s$, given as the unit complex number $e^{2\pi \mathrm{i}/3}$, that is, the unit tangent vector at 120° from the first frame vector t_s. The two coordinates of s are then not unknowns, and their two equations of (10.1) are replaced by the prescribed values: in the matrix, the rows $2s$ and $2s + 1$ become rows of the identity, and in the right-hand side, the entries become $\bar{u}_s$. This *row replacement* is `PreFactor`, which copies every unconstrained row and writes 1 on the diagonal of each constrained row before the LU factorization (`solver.compute`). Every other entry of the right-hand side is zero, $\mathrm{u}^0 = 0$ away from the source, which is the problem (4.5).

Remark 10.1 (Which vertices are constrained). *The loop in* `main` *that chooses the source scans the vertices and stops (*`break`*) at the first one that is either*

near the center of the model or above the height 0.83*. Exactly one vertex is constrained.*

Remark 10.2 (Symmetric elimination). *Row replacement makes the matrix non-symmetric even if it was symmetric. The symmetric alternative eliminates the constrained unknowns: move their columns, multiplied by the prescribed values, to the right-hand side, and keep only the rows and columns of the free unknowns. For (6.5), the reduced matrix is a principal submatrix of* $\mathrm{M} + \Delta t\,\mathrm{L}$*, hence symmetric positive definite.*

10.3. From the solution to vectors

The solve (`solver.solve(right_hand_side)`) returns u, two coordinates per vertex. Its magnitudes decay away from the source; its directions are the transported source vector. So the code keeps only the direction: reading u_i as the complex number $u_{i,1} + \mathrm{i}u_{i,2}$, it normalizes it, $\hat{u}_i = u_i/|u_i|$, takes its angle $\alpha_i = \arg \hat{u}_i$ (`log(R_i).imag()`), and rotates the first frame vector t_i by α_i about the vertex normal n_i, which is the tangent vector $T_i\hat{u}_i = \cos\alpha_i\, t_i + \sin\alpha_i\, b_i$. That vector is drawn as an arrow at the vertex (`display`).

10.4. What the program computes

Figure 15 shows the result on the cactus that `FEM.cpp` loads: one vector prescribed at one vertex has been spread over the whole surface, following its arms and trunk. To see that the directions are right we need a surface where parallel transport is known exactly.

Worked example 10.3. *On the unit sphere, the geodesics from a point s are the great circles through it, and transporting a vector along the great circle from s to x is the smallest rotation taking s to x, applied to the vector: the rolling of Section 14, with the normal turning from s to x in their common plane. The program* `ga_fem` *solves the problem of this text on a geodesic sphere of* 2,562 *vertices (an icosahedron subdivided four times, mean edge length* 0.076*), with the source at its top vertex, and compares the direction of the result at every vertex with that exact transport (Figure 16). Out to three quarters of the way to the antipode, the angle between the two is* 0.07° *on average and* 0.30° *at most. Near the antipode the error grows without bound, and it must: every great circle from s reaches the antipode, each with a different transported vector, so there is no shortest path and no correct answer there. The magnitude* $|u_i|$ *decays by twenty orders of magnitude from the source to the antipode, the screening of (4.5) over a distance of order* $\sqrt{\Delta t} = \bar{h}$*, and yet the direction, which is all we keep, is accurate wherever it is defined.*

11. Summary: From the Equations to the Code

Algorithm 1 lists the steps of the program, and Table 1 the formula behind every function. Read from top to bottom, the table is the derivation of this text in the order the code executes it: the geometry first, then the connection,

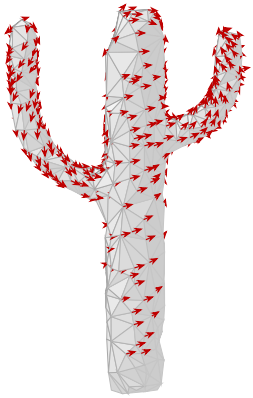


FIGURE 15. The output of the method on the cactus of `FEM.cpp` (502 vertices): the normalized solution at every visible vertex, drawn as a tangent vector.

Algorithm 1 Parallel transport of a tangent vector by one step of the vector heat equation (`main` of `FEM.cpp`).

1: Vertex normals (2.3), vertex frames T_i (2.4), face normals, areas and frames T_f (2.2)
2: $\Delta t \leftarrow \bar{h}^2$, the square of the mean edge length
3: choose the source vertex s and its vector $\bar{u}_s$; $\mathrm{b} \leftarrow 0$, $\mathrm{b}_s \leftarrow \bar{u}_s$
4: **for** every face f **do**
5: $\quad B \leftarrow [x_2 - x_1,\ x_3 - x_1]$, $\quad \nabla\phi_a \leftarrow B(B^TB)^{-1}\hat{\nabla}\hat{\phi}_a$, $\quad g_a^f \leftarrow T_f^T \nabla\phi_a$
6: $\quad R_a^f \leftarrow T_f^T\, Q_{a\to f}\, T_a$ (Rodrigues rotation $n_a \to n_f$)
7: $\quad$ add $\Delta t\, A_f\, (g_a^f \cdot g_b^f)\, (R_a^f)^T R_b^f$ to the block (a,b) of $\Delta t\,\mathrm{L}$, $\quad \frac{A_f}{3} I$ to the block (a,a) of M
8: **end for**
9: $\mathrm{A} \leftarrow I + \mathrm{M}^{-1}(\Delta t\,\mathrm{L})$; replace the rows of s by identity rows
10: solve $\mathrm{A\,u} = \mathrm{b}$ by sparse LU
11: for every vertex i: draw $T_i\, u_i/|u_i|$

then the element matrices, their assembly, the boundary condition, and the solve.

Three ideas carry the whole construction. The weak form trades the second derivatives of the heat equation for first derivatives of a trial field and a test field, so that piecewise linear fields can be used. Piecewise linear *tangent* fields exist only once the vectors of the three vertices of a face are brought into the plane of the face, and that is done by the discrete Levi-Civita connection, a 2×2 rotation per vertex and face. And because a face is flat, the covariant derivative inside it is an ordinary derivative, so that the stiffness of the connection Laplacian is the stiffness of the scalar Laplacian, multiplied on every edge by the transport between its two vertices.

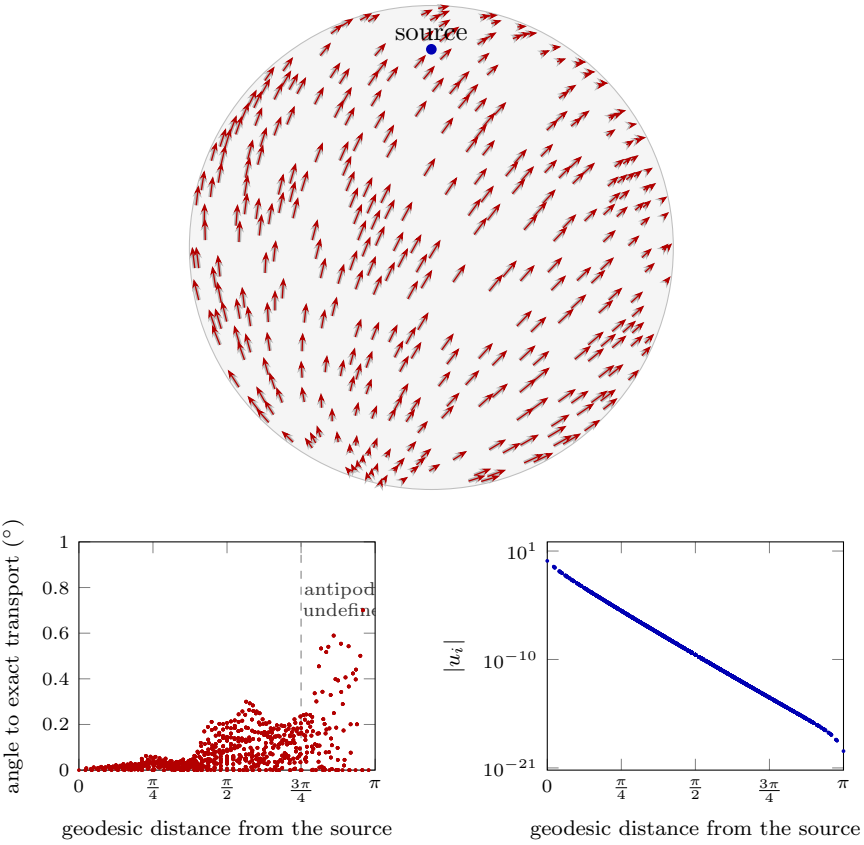


FIGURE 16. Validation on the sphere (Example 10.3). Top: the computed directions (thin red) over the exact parallel transport along great circles from the source (thick gray), on the visible half of the sphere. Bottom: the angle between the two at every vertex against the geodesic distance from the source (beyond the dashed line, three quarters of the way, the errors grow towards the antipode, where transport is not defined, and are not shown), and the magnitude of the solution, on a logarithmic scale.

Part 2. The Same Derivation in Geometric Algebra, and What Is Intrinsic

Part 1 worked with matrices: frames as 3×2 arrays, transports as 2×2 rotations, coordinates as pairs of numbers. Every one of those objects is a geometric object in disguise: a plane, a rotation, a direction in a plane. This part repeats the derivation with the objects themselves, in the geometric algebra $\mathbb{G}_3$ of space [2,3], and uses the clearer language to answer a question Part 1 left open: which parts of the computation are *intrinsic*, that is, depend

TABLE 1. Every formula and the function of `FEM.cpp` that evaluates it.

Quantity	Equation	Function
vertex normal n_i	(2.3)	`Mesh::computeNormals`
vertex frame T_i	(2.4)	`computeVertexTangentSpace`, `project`
face normal n_f, area A_f	(2.2)	`computeFaceNormals`
face frame T_f	Section 2	`computeFaceTangentSpace`
smallest rotation $Q_{i\to f}$	(3.4)	`alignVectors`, `bracket`
discrete connection R_i^f	(3.5)	`Rvf`
vertex-to-vertex transport (variant)	(3.6)	`Rvv`
edge matrix B, $B(B^TB)^{-1}$	(7.3), Prop. 7.1	`B`, `Binv`
gradients $\nabla\phi_a$, g_a^f	(7.4), (7.5)	`gradN`, `grad_i`
element stiffness L^f	(8.1)	`AssembleStiffnessElement-Embedded2`
lumped mass M	(8.3)	`AssembleDiagonalMassMatrix`
assembly of $\Delta t\,\mathrm{L}$	(9.1), (9.2)	`AssembleMatrix2`
time step $\Delta t = \bar{h}^2$	Section 4	`computeEdgeLengths`
$I + \Delta t\,\mathrm{M}^{-1}\mathrm{L}$	(10.1)	`IplusMinvTimesA`
Dirichlet rows, LU factorization	Section 10	`PreFactor`
normalization and drawing	Section 10	`display`

only on the curved surface as its inhabitants experience it, and which depend on how the surface sits in space.

12. The Tangent Plane as a Bivector

12.1. Geometric algebra in brief

In $\mathbb{G}_3$, vectors are multiplied with the geometric product $ab = a \cdot b + a \wedge b$, the sum of the inner product, a scalar, and the outer product, a *bivector*: an oriented plane element, with the orientation of the turn from a to b and magnitude the area of the parallelogram they span. Orthogonal vectors anticommute, parallel ones commute, and $aa = |a|^2$. The unit pseudoscalar $I = e_1e_2e_3$ squares to -1, commutes with everything, and turns a vector into the bivector of its orthogonal plane: $a \wedge b = I(a \times b)$. The reverse $\widetilde{X}$ of a product reverses the order of its vector factors. A *rotor* is an even element R with $R\widetilde{R} = 1$; it rotates any multivector by the sandwich $X \mapsto RX\widetilde{R}$, and the rotor that turns by the angle θ in the plane of the unit bivector B (in the sense of B) is $R = e^{-B\theta/2} = \cos\frac{\theta}{2} - B\sin\frac{\theta}{2}$.

12.2. The tangent bivector

At each point of the surface, instead of the normal n, take its dual, the unit bivector

$$\mathsf{T} = In, \tag{12.1}$$

the *tangent bivector*: the tangent plane itself, as an oriented plane element. Its square is $\mathsf{T}^2 = InIn = I^2n^2 = -1$. A vector a is tangent exactly when $a \wedge \mathsf{T} = 0$, and its projection onto the tangent plane and its rejection from it are

$$P(a) = (a \cdot \mathsf{T})\,\mathsf{T}^{-1} = -(a \cdot \mathsf{T})\,\mathsf{T}, \qquad a - P(a) = (a \wedge \mathsf{T})\,\mathsf{T}^{-1} = (a \cdot n)\,n, \tag{12.2}$$

which is the matrix $TT^T = I - nn^T$ of (2.1), now written without a choice of frame.

12.3. The tangent plane is the complex plane

The bivector T acts on tangent vectors by right multiplication. For a tangent vector a, $a \cdot n = 0$, so $an = a \wedge n$ and $a\mathsf{T} = aIn = I(an) = I(a \wedge n) = I^2(a \times n) = n \times a$:

$$a\,\mathsf{T} = n \times a, \tag{12.3}$$

the vector a turned by a quarter turn, counterclockwise about n, within the tangent plane. Applying it twice gives $a\mathsf{T}^2 = -a$. So the even elements $\alpha + \beta\mathsf{T}$ of the plane behave exactly like complex numbers, with T as the imaginary unit, and they act on tangent vectors by right multiplication:

$$a\,e^{\mathsf{T}\theta} = a\cos\theta + (n \times a)\sin\theta, \tag{12.4}$$

the rotation of a by θ about n (Figure 17). This is why Part 1 could read the coordinates u_i as complex numbers: with the first frame vector t, every tangent vector is

$$U = t\,\psi, \qquad \psi = u_1 + u_2\,\mathsf{T}, \tag{12.5}$$

since $t\mathsf{T} = n \times t = b$. We call ψ the *spinor coordinate* of U in the frame t: a complex number of the plane, $|\psi| = |U|$, and $\arg\psi$ is the angle from t to U. The pair (u_1, u_2) of the code is ψ written in the basis $(1, \mathsf{T})$. The frame is now a single unit vector t, the second vector $b = t\mathsf{T}$ being determined by the plane; and changing the frame, $t \to te^{\mathsf{T}\theta}$, changes the coordinate to $\psi \to e^{-\mathsf{T}\theta}\psi$.

13. Rotors Between Tangent Planes

13.1. The smallest rotor taking one normal to another

Proposition 13.1. *For unit vectors a and c with $a \cdot c \neq -1$,*

$$Q = \frac{1 + c\,a}{|1 + c\,a|} = \frac{1 + c\,a}{\sqrt{2(1 + a \cdot c)}} \tag{13.1}$$

is the rotor that turns a to c in the plane $a \wedge c$, by their angle; it is the rotation of the Rodrigues formula (3.4). It takes the bivector Ia to Ic.

Proof. $ca = c \cdot a + c \wedge a$ is the product of two unit vectors, a rotor up to normalization that turns by twice their angle ϕ. The half-way vector $m = (a + c)/|a + c|$ splits that angle in two, and cm, the product of two unit vectors at the angle $\phi/2$, is the rotor that turns by ϕ from a through m to c: $cm\,a\,mc = c(mam)c = c\,c\,c = c$, because reflecting a in the line of m gives c.

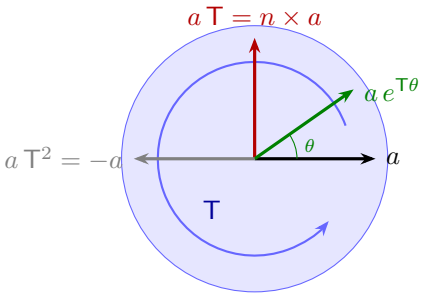


FIGURE 17. The tangent bivector $\mathsf{T} = In$ as an oriented disk in the tangent plane. Right multiplication by T turns a tangent vector a by a quarter turn in the sense of T, twice gives $-a$, and $e^{\mathsf{T}\theta} = \cos\theta + \mathsf{T}\sin\theta$ turns it by θ: the even elements of the plane are the complex numbers, with T as imaginary unit.

Now $cm = c(a+c)/|a+c| = (1+ca)/|a+c|$, and $|a+c|^2 = 2(1+a\cdot c)$. The plane of the rotation is that of $c \wedge a$, so its axis is $a \times c$, as in (3.4). Finally a rotor commutes with I, so $Q(Ia)\widetilde{Q} = I(Qa\widetilde{Q}) = Ic$. □

So the rotor

$$Q_{i\to f} = \frac{1 + n_f\, n_i}{|1 + n_f\, n_i|} \tag{13.2}$$

takes the normal of vertex i to that of face f and, by the last statement, the tangent bivector $\mathsf{T}_i = In_i$ to $\mathsf{T}_f = In_f$: it carries the tangent plane of the vertex onto that of the face, and with it every tangent vector, $a \mapsto Q_{i\to f}\, a\, \widetilde{Q}_{i\to f}$. The failure of `alignVectors` for opposite normals is visible here: $1 + n_f n_i = 0$ when $n_f = -n_i$, and no plane of rotation is singled out.

Worked example 13.2. *For the vertex and the face of Example 3.3,* $n_f n_i = n_f \cdot n_i + n_f \wedge n_i = \frac{1}{\sqrt{2}} + I(n_f \times n_i)$ *with* $n_f \times n_i = (-\frac{1}{\sqrt{2}}, 0, 0)$. *Normalizing* $1 + n_f n_i$, *whose norm is* $\sqrt{2(1+1/\sqrt{2})} = 1.8478$,

$$Q = 0.9239 - 0.3827\, Ie_1 = \cos 22.5^\circ - Ie_1 \sin 22.5^\circ = e^{-Ie_1\pi/8},$$

the rotor of the rotation by 45° *in the plane* $Ie_1 = e_2e_3$, *about the* x *axis, as found with matrices (the program prints* $Q = 0.923880 + I(-0.382683, 0, 0)$*). It takes* $t_i = e_2$ *to* $c = Qe_2\widetilde{Q} = (e_2 + e_3)/\sqrt{2}$, *and the connection is the geometric product* $t_f c = e_1(e_2+e_3)/\sqrt{2} = (e_1e_2 + e_1e_3)/\sqrt{2}$, *a unit bivector with no scalar part. Since* $\mathsf{T}_f = In_f = (-Ie_2 + Ie_3)/\sqrt{2} = (-e_3e_1 + e_1e_2)/\sqrt{2} = (e_1e_3 + e_1e_2)/\sqrt{2}$, *this is* $t_f c = \mathsf{T}_f = e^{\mathsf{T}_f\pi/2}$: $\rho_i^f = 90^\circ$, *the same number, now read off a single product.*

13.2. The connection angle

The transported first frame vector $Qt_i\widetilde{Q}$ and the face frame vector t_f are two unit vectors of the face plane. Their geometric product is a complex number of that plane:

$$e^{\mathsf{T}_f \rho_i^f} = t_f \left(Q_{i\to f}\, t_i\, \widetilde{Q}_{i\to f}\right), \tag{13.3}$$

since for unit vectors t_f and c in the plane, $t_f c = t_f \cdot c + t_f \wedge c = \cos\rho + \mathsf{T}_f \sin\rho$, with ρ the angle from t_f to c (the bivector $t_f \wedge c$ is $\sin\rho$ times $t_f \wedge b_f = \mathsf{T}_f$). This single number is the discrete connection: transporting $U = t_i\psi$ from vertex i into face f gives

$$Q\, t_i \psi\, \widetilde{Q} = (Qt_i\widetilde{Q})(Q\psi\widetilde{Q}) = t_f\, e^{\mathsf{T}_f \rho_i^f}\, \psi_f, \tag{13.4}$$

where $\psi_f = u_1 + u_2\mathsf{T}_f$ is the same pair of numbers read in the face plane (because $Q\mathsf{T}_i\widetilde{Q} = \mathsf{T}_f$). In spinor coordinates, transport from the vertex to the face is *multiplication by the unit complex number* $e^{\mathsf{T}_f\rho_i^f}$, and the 2×2 matrix R_i^f of (3.5) is the matrix of that multiplication in the basis $(1, \mathsf{T}_f)$:

$$R_i^f = \begin{pmatrix} \cos\rho_i^f & -\sin\rho_i^f \\ \sin\rho_i^f & \cos\rho_i^f \end{pmatrix}. \tag{13.5}$$

The transport between two vertices through the face, $(R_i^f)^T R_j^f$, is multiplication by $e^{\mathsf{T}(\rho_j^f - \rho_i^f)}$: the connection angles subtract, and the arbitrary angle of the face frame cancels, as found in Section 3.

14. Intrinsic and Extrinsic Geometry

14.1. What an inhabitant of the surface can measure

A two-dimensional being living on the surface can measure lengths of curves and angles between them, and nothing else: it cannot see the normal, nor how the surface bends in space. Everything that can be computed from lengths and angles alone is *intrinsic*; it is unchanged if the surface is bent without stretching, as a sheet of paper rolled into a cylinder. Everything else, the normal n, the tangent bivector T as an element of $\mathbb{G}_3$, the bending of the surface, is *extrinsic*. Gauss's *Theorema Egregium* says that the Gaussian curvature K, defined extrinsically as the product of the principal curvatures, is intrinsic.

The questions this method answers are intrinsic: parallel transport, the covariant derivative, the Dirichlet energy, the connection Laplacian, and hence the heat flow and the transported field, depend only on the metric. A paper cylinder rolled from a flat sheet transports vectors as the flat sheet does, and the method should give the same field on both. The computation, however, uses extrinsic tools: normals, rotations of $\mathbb{R}^3$, projections. The rest of this section separates the two.

14.2. The Gauss formula

Let a be a tangent direction and b a tangent field, and differentiate b in $\mathbb{R}^3$: $a \cdot \nabla b$. Split the result into its tangent and normal parts with (12.2):

$$a \cdot \nabla b = \underbrace{P(a \cdot \nabla b)}_{\nabla_a b} + \underbrace{\big((a \cdot \nabla b) \cdot n\big)\, n}_{\mathrm{II}(a,b)\, n}. \tag{14.1}$$

The tangent part is the covariant derivative (3.1); the normal part is the *second fundamental form*, $\mathrm{II}(a,b) = (a \cdot \nabla b) \cdot n = -b \cdot (a \cdot \nabla n)$ (differentiate $b \cdot n = 0$), which measures how the surface bends under b: it is extrinsic. The covariant derivative is the remaining, intrinsic part. That it depends only on the metric is the content of the fundamental theorem of Riemannian geometry: there is exactly one connection on tangent fields that is compatible with the metric, (3.2), and torsion free, $\nabla_a b - \nabla_b a = [a,b]$; it is given by the metric alone (the Koszul formula), and the projection $\nabla_a b = P(a \cdot \nabla b)$ has both properties [1].

14.3. Parallel transport is rolling without twisting

How does a tangent plane move along a curve $\gamma(s)$ if it carries its vectors parallel? The plane must follow the surface, turning $n(s)$ along with it, but it should not spin about the normal. In geometric algebra this is a rotor equation.

Proposition 14.1. *Let $R(s)$ solve $R' = \frac{1}{2}\Omega\, R$ with $R(0) = 1$, where $\Omega = n' \wedge n$ and $n(s)$ is the normal along the curve. Then $Rn(0)\widetilde{R} = n(s)$, and for every tangent vector a_0 at $\gamma(0)$, the field $a(s) = R\, a_0 \widetilde{R}$ is tangent and parallel along the curve.*

Proof. For $v(s) = Rv_0\widetilde{R}$, the product rule and $\widetilde{R}' = -\frac{1}{2}\widetilde{R}\,\Omega$ (because Ω is a bivector, $\widetilde{\Omega} = -\Omega$) give $v' = \frac{1}{2}(\Omega v - v\Omega) = \Omega \cdot v$, the inner product of a bivector and a vector. With $\Omega = n' \wedge n$ and the rule $(x \wedge y) \cdot v = x(y \cdot v) - y(x \cdot v)$,

$$\Omega \cdot v = n'(n \cdot v) - n(n' \cdot v).$$

First, $m(s) = Rn(0)\widetilde{R}$ satisfies $m' = n'(n \cdot m) - n(n' \cdot m)$, and $m = n$ is a solution, since $n \cdot n = 1$ and $n' \cdot n = 0$; by uniqueness $m = n$. So R takes the normal along the curve, hence the tangent planes onto each other, and $a(s)$ stays tangent. Second, for the tangent field a, $n \cdot a = 0$, so $a' = -(n' \cdot a)\, n$: the derivative of a is purely normal, its tangent part vanishes, and $\nabla_{\gamma'} a = P(a') = 0$. □

Figure 18 shows the rolling planes along a curve on a sphere. The bivector $\Omega = n' \wedge n$ contains the normal, so it never rotates within the tangent plane: the plane *rolls* along the curve, as a coin rolls on a table, without twisting about its normal. Parallel transport is this rolling. The discrete transport $Q_{i \to f}$ of (13.2) is the same idea over a finite step: the rotor in the plane $n_f \wedge n_i$, which contains both normals and therefore has no component of spin about either of them.

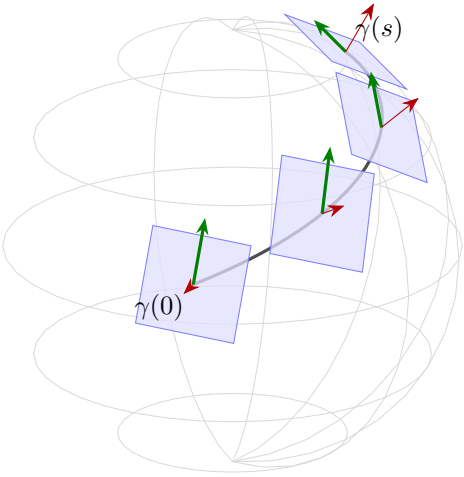


FIGURE 18. Parallel transport is rolling without twisting. Along a curve γ on a sphere, the tangent plane (blue) is rolled by the rotor of Proposition 14.1, here integrated as a sequence of smallest rotations between consecutive normals (red). The transported vector (green) changes only in the direction of the normal, so its covariant derivative vanishes.

14.4. What is intrinsic in the discrete transport

On a triangle mesh the intrinsic geometry is that of the flat faces glued along their edges: it is given by the edge lengths. Transport *across an edge*, from one face to the next, is intrinsic: by Proposition 13.1 with the two face normals, the rotor Q turns about the shared edge (the direction of $n_f \times n_g$), which is exactly the unfolding of one face onto the plane of the other, an isometry that depends only on the two triangles and not on the angle at which they meet in space.

A vertex is different. It has no tangent plane: it is the apex of a cone whose total angle $\Theta_i = \sum_f \alpha_{i,f}$, the sum of the angles of the incident faces at the vertex, is in general not 2π. The code gives the vertex a plane anyway, the plane orthogonal to the vertex normal (2.3), and transports between that plane and the faces with the rolling rotor $Q_{i\to f}$. This is an *extrinsic* choice: bending the mesh at the vertex without stretching any triangle changes the vertex normal and the rotors, although no length changes. The error it introduces vanishes as the mesh is refined on a smooth surface, because the vertex normal then converges to the normal of the smooth surface and the rolling rotors to its parallel transport. An intrinsic alternative [4, 9] defines the tangent space of the vertex by the angles around it, rescaled by $2\pi/\Theta_i$ so that they close up, and the transport to a face, or across an edge, by those rescaled angles; in the code it would replace only the computation of the connection angle ρ_i^f (`Rvf`), leaving the finite element machinery untouched.

14.5. Holonomy and curvature

Carry a vector parallel around a small closed loop and it comes back rotated, in the tangent plane, by an angle: the *holonomy* of the loop. For rolling, Proposition 14.1, the angle has a geometric meaning: it is the area enclosed by the loop that the normal $n(s)$ traces on the unit sphere. By the Gauss–Bonnet theorem that area is $\int K\, dA$ over the region enclosed by the loop on the surface, which is why holonomy measures curvature and is intrinsic. On a mesh, the intrinsic transport around a vertex, unfolding the faces one after another, closes with the rotation $2\pi - \Theta_i$, the *angle defect*, the discrete total curvature of the vertex. The code's transport, which rolls through the vertex normals, closes with the area of the spherical polygon whose corners are the normals visited by the loop, those of the neighboring vertices and faces, which approximates the angle defect on fine meshes; the remark of Section 3 should be read in this sense.

15. The Covariant Derivative as a Connection on Spinor Coordinates

15.1. The connection form

Choose a frame field, a unit tangent vector $t(x)$ at each point, and write fields as $U = t\psi$ with spinor coordinates $\psi = u_1 + u_2\mathsf{T}$. Differentiating along a tangent direction a, $a\cdot\nabla U = (a\cdot\nabla t)\,\psi + t\,(a\cdot\nabla\psi)$. The covariant derivative of the frame vector is tangent and orthogonal to t (since $|t| = 1$), so it is a multiple of $t\mathsf{T}$:

$$\nabla_a t = \omega(a)\, t\,\mathsf{T}, \tag{15.1}$$

where the scalar $\omega(a)$, linear in a, is the rate at which the frame turns, measured against parallel transport: the *connection form* of the frame, the angle $\theta(a)$ of (3.3). Writing $U = u_1 t + u_2 b$ with $b = t\mathsf{T}$, and using also $\nabla_a b = -\omega(a)\, t$ (the frame turns rigidly, since $t\cdot b = 0$), the covariant derivative of U is

$$\nabla_a(t\psi) = t\left(a\cdot\nabla\psi + \omega(a)\,\mathsf{T}\,\psi\right) = t\,D_a\psi, \qquad D_a = a\cdot\nabla + \omega(a)\,\mathsf{T}, \tag{15.2}$$

where $a\cdot\nabla\psi = (a\cdot\nabla u_1) + (a\cdot\nabla u_2)\,\mathsf{T}$ differentiates the two coefficients, and $\omega\mathsf{T}\psi = \omega(u_1\mathsf{T} - u_2)$ collects the terms $u_1\nabla_a t + u_2\nabla_a b$. On spinor coordinates, the Levi-Civita connection is the ordinary derivative plus a multiplication by $\omega\mathsf{T}$: an infinitesimal rotation. In the language of physics, ψ is a charged field and $\omega\mathsf{T}$ a $U(1)$ gauge potential; the connection Laplacian is the corresponding *magnetic* Laplacian, and the curvature of the connection, the "magnetic field", is $d\omega$, the Gaussian curvature times the area form [4].

15.2. Frame independence

The frame t is arbitrary, and nothing physical may depend on it. Rotating the frame by a field of angles, $t \to t' = t\,e^{\mathsf{T}\theta}$, changes the coordinates to

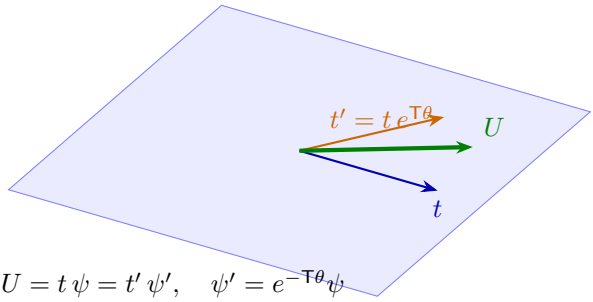


FIGURE 19. A change of frame is a change of gauge. The same tangent vector U has the spinor coordinate ψ in the frame t and $\psi' = e^{-\mathsf{T}\theta}\psi$ in the frame $t' = t\,e^{\mathsf{T}\theta}$; nothing geometric depends on the choice.

$\psi' = e^{-\mathsf{T}\theta}\psi$ and, since $\nabla_a t' = (\nabla_a t)\,e^{\mathsf{T}\theta} + t\,(a\cdot\nabla\theta)\,\mathsf{T}e^{\mathsf{T}\theta}$, the connection form to $\omega' = \omega + d\theta$; and then

$$D'_a\psi' = e^{-\mathsf{T}\theta}\,D_a\psi, \tag{15.3}$$

as one checks by differentiating $e^{-\mathsf{T}\theta}\psi$: the extra term $-(a\cdot\nabla\theta)\,\mathsf{T}\,e^{-\mathsf{T}\theta}\psi$ is cancelled by the extra term $(a\cdot\nabla\theta)\,\mathsf{T}\,e^{-\mathsf{T}\theta}\psi$ of ω'. So $\nabla U = t\,D\psi = t'D'\psi'$ is the same tangent object in both frames, $|D\psi|$ is frame independent, and so are the Dirichlet energy, the connection Laplacian and the solution of the heat equation. In the discrete setting this is the cancellation of the face frame in $(R_i^f)^T R_j^f$ and the statement that changing a vertex frame t_i rotates the computed u_i by the opposite angle and changes nothing else: the arrow $t_i\hat{\psi}_i$ drawn by the code is independent of the choice of t_i in `computeVertexTangentSpace`.

15.3. The energy and the equation in spinor form

With $|U| = |\psi|$ and $|\nabla_a U| = |D_a\psi|$,

$$E = \frac{1}{2}\int_\Omega \sum_k |D_{e_k}\psi|^2\,dA, \qquad \frac{\partial\psi}{\partial t} = -D^*D\,\psi, \tag{15.4}$$

and the backward Euler step and its weak form, (4.4) and (5.1), read

$$\int_\Omega \langle\psi\,\widetilde{\chi}\rangle\,dA + \Delta t\int_\Omega \sum_k \langle (D_{e_k}\psi)\,\widetilde{(D_{e_k}\chi)}\rangle\,dA = \int_\Omega \langle\psi^0\,\widetilde{\chi}\rangle\,dA \tag{15.5}$$

for every test spinor χ, where $\langle\psi\widetilde{\chi}\rangle$ is the scalar part of $\psi\widetilde{\chi}$: for $\psi = u_1 + u_2\mathsf{T}$ and $\chi = v_1 + v_2\mathsf{T}$, $\widetilde{\chi} = v_1 - v_2\mathsf{T}$ and $\langle\psi\widetilde{\chi}\rangle = u_1v_1 + u_2v_2 = U\cdot V$. This is the real part of $\psi\bar{\chi}$, the familiar Hermitian inner product, with the reverse playing the role of complex conjugation.

16. The Finite Element Method in Geometric Algebra

16.1. Spinor hat functions

The trial field (6.1), in spinor coordinates of the face, is

$$U\big|_f = t_f\,\psi_f, \qquad \psi_f = \sum_{i\in f} \phi_i\; e^{\mathsf{T}_f \rho_i^f}\;\psi_i, \tag{16.1}$$

by (13.4): each vertex coordinate is turned by its connection angle into the face, and the three are interpolated with the hats. The face is flat and t_f constant, so $\omega = 0$ on the face and $D = \nabla$, and

$$D_a\psi_f = \sum_{i\in f} (a\cdot\nabla\phi_i)\, e^{\mathsf{T}_f \rho_i^f}\;\psi_i. \tag{16.2}$$

16.2. The gradients of the hat functions: the reciprocal frame

On the face with vertices x_1, x_2, x_3, take the *edge frame* $e_1 = x_2 - x_1$, $e_2 = x_3 - x_1$, the columns of B in (7.3), and their *area bivector* $E = e_1 \wedge e_2 = 2A_f\,\mathsf{T}_f$. The *reciprocal frame* e^1, e^2 is the pair of vectors of the face plane with $e^k \cdot e_m = \delta_{km}$. In geometric algebra it is written directly:

$$e^1 = e_2 \cdot E^{-1}, \qquad e^2 = -\,e_1 \cdot E^{-1}, \qquad E^{-1} = -\frac{E}{|E|^2}. \tag{16.3}$$

Indeed $e_2 \cdot E$ lies in the plane and is orthogonal to e_2, and $(e_2\cdot E)\cdot e_1 = (e_2\cdot e_1)^2 - |e_2|^2|e_1|^2 = -|E|^2$, so $e^1\cdot e_1 = 1$ and $e^1 \cdot e_2 = 0$; similarly for e^2.

Proposition 16.1. *The gradients of the hat functions of the face are $\nabla\phi_2 = e^1$, $\nabla\phi_3 = e^2$ and $\nabla\phi_1 = -e^1 - e^2$; the reciprocal frame is the pair of columns of $B(B^TB)^{-1}$,* `Binv`*. Equivalently, for each vertex a with the opposite edge $x_c - x_b$ (in the counterclockwise order a, b, c),*

$$\nabla\phi_a = \frac{(x_c - x_b)\,\mathsf{T}_f}{2A_f}\,: \tag{16.4}$$

the opposite edge, turned a quarter turn inward, divided by twice the area.

Proof. The gradient g of ϕ_a lies in the plane and is determined by its inner products with the edges, $g\cdot e_m = \partial\hat{\phi}_a/\partial\xi_m$ (proof of Proposition 7.1). Writing $g = \sum_k (\partial\hat{\phi}_a/\partial\xi_k)\, e^k$ satisfies these equations, by the defining property of the reciprocal frame, and with (7.2) gives the three gradients. The matrix $B(B^TB)^{-1}$ has the same defining property, $B^T\big(B(B^TB)^{-1}\big) = I$, and its columns lie in the plane, so its columns are e^1 and e^2. For (16.4), take $a = 2$; in the counterclockwise order $a, b, c = 2, 3, 1$ its opposite edge is $x_c - x_b = x_1 - x_3 = -e_2$. Since $E = 2A_f\mathsf{T}_f$ and $\mathsf{T}_f^{-1} = -\mathsf{T}_f$, $E^{-1} = -\mathsf{T}_f/(2A_f)$, and by (16.3) $e^1 = e_2\cdot E^{-1} = -e_2\mathsf{T}_f/(2A_f) = (x_1 - x_3)\,\mathsf{T}_f/(2A_f)$, using that for a vector in the plane of a bivector the inner and geometric products coincide. The other vertices follow by relabeling. (On the triangle $x_1 = 0$, $x_2 = e_x$, $x_3 = e_y$ with $n = e_z$: $(x_1 - x_3)\mathsf{T} = n \times (-e_y) = e_x$, which is indeed $\nabla\phi_2$.) $\square$

The formula (16.4) makes the cotangent formula of Proposition 7.3 immediate: the quarter turn preserves inner products, so $A_f\,\nabla\phi_a\cdot\nabla\phi_b = \frac{(x_c-x_b)\cdot(x_a-x_c)}{4A_f}$, and for the two edges meeting at c, the ratio of their inner product to $|(x_c-x_b)\wedge(x_a-x_c)| = 2A_f$ is $-\cot\theta_c$. In the vector manifold calculus of Hestenes and Sobczyk [3], the reciprocal frame is the general rule: the vector derivative on a face is $\nabla = e^1\partial_{\xi_1} + e^2\partial_{\xi_2}$.

Worked example 16.2. *For the triangle of Example 7.2, $E = e_1\wedge e_2 = I(e_1\times e_2) = I(0,-2,2)$, with $|E| = 2\sqrt{2} = 2A_f$, and $\mathsf{T}_f = E/|E| = In_f$. The gradient of ϕ_2 by (16.4) is the opposite edge $x_1 - x_3 = (0,-1,-1)$, turned a quarter turn in the face, $(x_1-x_3)\,\mathsf{T}_f = n_f\times(x_1-x_3) = \frac{1}{\sqrt{2}}(0,-1,1)\times(0,-1,-1) = \frac{1}{\sqrt{2}}(2,0,0)$, divided by $2A_f = 2\sqrt{2}$: $\nabla\phi_2 = (\frac{1}{2},0,0)$, as in Example 7.2, without inverting any matrix.*

16.3. The element matrices

For two vertices i,j of the face, the stiffness integrand of (15.5) with the trial field (16.2) and the test spinor $\chi = \phi_i\, e^{\mathsf{T}_f\rho_i^f}\,\chi_i$ is

$$\sum_k \big\langle (D_{e_k}\psi)\,\widetilde{(D_{e_k}\chi)}\big\rangle = (\nabla\phi_i\cdot\nabla\phi_j)\,\big\langle e^{\mathsf{T}_f\rho_j^f}\psi_j\,\widetilde{\chi_i}\,e^{-\mathsf{T}_f\rho_i^f}\big\rangle$$
$$= (\nabla\phi_i\cdot\nabla\phi_j)\,\big\langle e^{\mathsf{T}_f(\rho_j^f-\rho_i^f)}\,\psi_j\,\widetilde{\chi_i}\big\rangle,$$

using that the even elements of the plane commute and the cyclic property of the scalar part. So, as a complex matrix,

$$\mathrm{L}_{ij}^f = A_f\,(\nabla\phi_i\cdot\nabla\phi_j)\,e^{\mathsf{T}(\rho_j^f-\rho_i^f)}, \qquad \mathrm{M}_{ii}^f = \frac{A_f}{3}: \tag{16.5}$$

one complex number per pair of vertices instead of a 2×2 block. The global stiffness L is then an $n\times n$ *Hermitian* matrix, $\mathrm{L}_{ji} = \overline{\mathrm{L}_{ij}}$ (the reverse of $e^{\mathsf{T}\rho}$ is $e^{-\mathsf{T}\rho}$), the form in which discrete connection Laplacians are usually written [4, 9], and the backward Euler step is the complex system

$$(\mathrm{M}+\Delta t\,\mathrm{L})\,\psi = \mathrm{M}\,\psi^0, \qquad \psi\in\mathbb{C}^n. \tag{16.6}$$

16.4. The code solves the real form of the complex system

A complex number $z = \alpha+\beta\mathsf{T}$ acts on $\psi = u_1+u_2\mathsf{T}$ by multiplication, and in the basis $(1,\mathsf{T})$ that multiplication is the real 2×2 matrix $\begin{pmatrix}\alpha & -\beta\\ \beta & \alpha\end{pmatrix}$. Replacing every complex entry of (16.6) by this matrix, and every complex unknown by its two real coordinates, turns the $n\times n$ complex system into a $2n\times 2n$ real one with the same solution: and that real system is exactly the one assembled in Part 1. The element block (6.7) is the matrix of multiplication by (16.5), $(R_i^f)^T R_j^f$ being the matrix of $e^{\mathsf{T}(\rho_j^f-\rho_i^f)}$ by (13.5), and the real mass and the Dirichlet rows are the real forms of their complex counterparts. The code works with real arithmetic and the general sparse solver; the complex form halves the size of the system and shows that every block of the code has the structure "scalar times rotation".

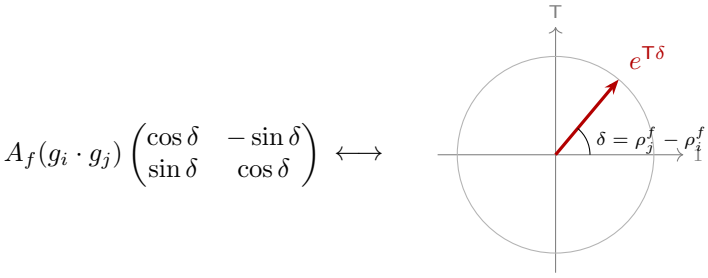


FIGURE 20. Every 2×2 block of the code is a real multiple of a rotation, the matrix of multiplication by a complex number $e^{\mathsf{T}\delta}$ of the tangent plane; the complex form stores only that number.

17. The Geometric Algebra in Code

The repository contains a second program, `ga_fem.cpp`, which needs no graphics and implements Part 2 with the small header `GAConnection.h`. It is built and run from `src/` with

```
c++ -std=c++17 -O2 -I<eigen> ga_fem.cpp HalfEdge/Mesh.cpp -o ga_fem
./ga_fem ../fem-basic-2d/data
```

It prints the worked examples of this text, solves the cactus of `FEM.cpp` twice, with the real 2×2 blocks of Part 1 and with the complex Hermitian system of Part 2, solves the sphere of Example 10.3, and writes the data of Figures 15 and 16. On the cactus, the two solutions agree to $6\cdot10^{-16}$, the rounding error of double precision: the two parts of this text compute the same thing.

The algebra. Only the grades the method needs are implemented: vectors, bivectors stored by their dual vectors ($B = Ib$), and rotors $w + Ib$. The product of two vectors is a rotor up to normalization, and the inner product of a vector with a bivector is the quarter turn (12.3):

```
19 using Vec3 = Eigen::Vector3d;
20 using Complex = std::complex<double>;
21
22 /// A bivector B = I b, stored by its dual vector b. Unit bivectors square to -1.
23 struct Bivector {
24     Vec3 b;
25     Bivector operator*(double s) const { return { b * s }; }
26     double norm() const { return b.norm(); }
27 };
28
29 /// The outer product a ^ c = I (a x c): the oriented plane of a and c.
30 inline Bivector wedge(const Vec3& a, const Vec3& c) { return { a.cross(c) }; }
31
32 /// The inner product of a vector and a bivector, a . B, a vector of the plane of B,
       orthogonal to a.
33 /// For B = I b: a . (I b) = I (a ^ b) = I I (a x b) = b x a.
34 inline Vec3 dot(const Vec3& a, const Bivector& B) { return B.b.cross(a); }
35
36 /// B^-1 = -B / |B|^2, since B^2 = -|B|^2.
37 inline Bivector inverse(const Bivector& B) { return B * (-1.0 / B.b.squaredNorm()); }
38
```

```
/// A rotor R = w + I b (Section 12). R a ~R rotates the vector a.
struct Rotor {
    double w;
    Vec3 b;
    /// R a ~R. With R = w + I b, the rotation is the one of the unit quaternion (w, -b):
    /// a' = a + 2 w (u x a) + 2 u x (u x a), u = -b.
    Vec3 rotate(const Vec3& a) const {
        const Vec3 u = -b;
        const Vec3 t = 2.0 * u.cross(a);
        return a + w * t + u.cross(t);
    }
};

/// The geometric product of two vectors, c a = c . a + c ^ a, a rotor up to normalization.
inline Rotor product(const Vec3& c, const Vec3& a) { return { c.dot(a), c.cross(a) }; }

/// The tangent bivector T = I n of a unit normal n (eq. ga_T): the tangent plane as an oriented plane element.
inline Bivector tangentBivector(const Vec3& n) { return { n }; }

/// a T for a tangent vector a: the quarter turn n x a (eq. ga_quarter_turn).
/// For a in the plane of T the geometric product equals the inner product, a T = a . T.
inline Vec3 quarterTurn(const Vec3& a, const Bivector& T) { return dot(a, T); }
```

The connection. The smallest rotor of Proposition 13.1, and the connection (13.3) as the geometric product of two unit vectors of the face plane, a complex number of that plane:

```
/// The smallest rotor taking the unit vector a to the unit vector c (Proposition ga_rotor):
/// Q = (1 + c a) / |1 + c a|. Undefined for c = -a, where 1 + c a = 0.
inline Rotor rotorBetween(const Vec3& a, const Vec3& c) {
    const Rotor ca = product(c, a);
    const double w = 1.0 + ca.w;
    const double norm = std::sqrt(w * w + ca.b.squaredNorm()); // |1 + c a| = sqrt(2 (1 + a.c))
    return { w / norm, ca.b / norm };
}

/// The complex number of the plane with normal n given by the product of two unit vectors of that plane:
/// t c = t . c + t ^ c = cos(rho) + T sin(rho), with t ^ c = (t x c . n) T (eq. ga_connection).
inline Complex planeProduct(const Vec3& t, const Vec3& c, const Vec3& n) {
    return { t.dot(c), t.cross(c).dot(n) };
}

/// The discrete connection from vertex i to face f, as a unit complex number (eq. ga_connection):
/// e^{T_f rho_i^f} = t_f (Q t_i ~Q), Q = rotorBetween(n_i, n_f).
/// Transporting the spinor coordinate psi of vertex i into the face is multiplication by it (eq. ga_transport).
inline Complex connection(const Vec3& t_i, const Vec3& n_i, const Vec3& t_f, const Vec3& n_f) {
    const Rotor Q = rotorBetween(n_i, n_f);
    return planeProduct(t_f, Q.rotate(t_i), n_f);
}
```

The gradients. The reciprocal frame of Proposition 16.1, and the single-vertex form (16.4):

```
/// The gradients of the three hat functions of the triangle x1, x2, x3 (Proposition ga_reciprocal):
/// the reciprocal frame e^1 = e2 . E^-1, e^2 = -e1 . E^-1 of the edges e1 = x2 - x1, e2 = x3 - x1, with
/// E = e1 ^ e2; then grad phi_2 = e^1, grad phi_3 = e^2, grad phi_1 = -e^1 - e^2.
inline void hatGradients(const Vec3& x1, const Vec3& x2, const Vec3& x3, Vec3 grad[3]) {
    const Vec3 e1 = x2 - x1, e2 = x3 - x1;
    const Bivector Einv = inverse(wedge(e1, e2));
    const Vec3 r1 = dot(e2, Einv);
    const Vec3 r2 = -dot(e1, Einv);
    grad[0] = -r1 - r2;
    grad[1] = r1;
    grad[2] = r2;
}

/// The same gradient, one vertex at a time (eq. ga_grad_edge): the opposite edge, from b to c in the
/// counterclockwise order a, b, c, turned a quarter turn in the plane of the face, divided by twice the area.
inline Vec3 hatGradient(const Vec3& xb, const Vec3& xc, const Bivector& Tf, double area) {
    return quarterTurn(xc - xb, Tf) / (2.0 * area);
}
```

The complex system. The assembly of (16.5), the scaling by the lumped mass, and the Dirichlet row, with `std::complex` entries. Compare it line by line with `AssembleStiffnessElementEmbedded2`, `AssembleMatrix2`, `IplusMinvTimesA` and `PreFactor`, where each complex number is a 2×2 block:

```
/// Section 15: L_ij += A_f (grad phi_i . grad phi_j) e^{T (rho_j - rho_i)}, M_ii += A_f / 3, then the same
/// scaling I + M^-1 (dt L) and the same Dirichlet row as the matrix version, now with complex numbers.
static Eigen::VectorXcd solve(Mesh& mesh, const Geometry& g, double dt, int source, Complex value) {
    const int n = mesh.numVertices();
    std::vector<double> mass(n, 0.0);
    std::map<std::pair<int, int>, Complex> A;
    for (Face& f : mesh.getFaces()) {
        Vertex* v[3];
        faceVertices(f, v);
        ga::Vec3 grad[3];
        ga::hatGradients(v[0]->p, v[1]->p, v[2]->p, grad); // the reciprocal frame
        const double area = g.faceArea[f.ID];
        Complex rho[3]; // e^{T_f rho_a^f}
        for (int a = 0; a < 3; ++a)
            rho[a] = ga::connection(g.vertexT[v[a]->ID], v[a]->n, g.faceT[f.ID], g.faceNormal[f.ID]);
        for (int a = 0; a < 3; ++a) {
            mass[v[a]->ID] += area / 3.0;
            for (int b = 0; b < 3; ++b) {
                // transport from b to a through the face: e^{T (rho_b - rho_a)} = e^{T rho_b} conj(e^{T rho_a})
                const Complex transport = a == b ? Complex(1.0) : rho[b] * std::conj(rho[a]);
                A[{ v[a]->ID, v[b]->ID }] += dt * area * grad[a].dot(grad[b]) * transport;
            }
        }
    }
    std::vector<Eigen::Triplet<Complex>> triplets;
    for (const auto& [ij, value_ij] : A) {
        if (ij.first == source) continue;
```

TABLE 2. The objects of Part 1 and of Part 2, and whether they depend only on the metric of the surface (intrinsic) or on its embedding in space (extrinsic).

Object	Matrices (Part 1)	Geometric algebra	Nature
tangent plane	$TT^T = I - nn^T$	bivector $\mathsf{T} = In$	extrinsic
frame	$T = [t \;\; n \times t]$	one vector t, $b = t\mathsf{T}$	a gauge choice
coordinates	$u \in \mathbb{R}^2$	spinor $\psi = u_1 + u_2\mathsf{T}$	gauge dependent
rotation in the plane	2×2 rotation	$e^{\mathsf{T}\theta}$	intrinsic
plane to plane	Rodrigues Q	rotor $(1 + n_f n_i)/\lvert 1 + n_f n_i\rvert$	extrinsic tool
discrete connection	$R_i^f = T_f^T Q T_i$	$e^{\mathsf{T}\rho_i^f} = t_f Q t_i \widetilde{Q}$	see Sec. 14
transport $j \to i$	$(R_i^f)^T R_j^f$	$e^{\mathsf{T}(\rho_j - \rho_i)}$	gauge covariant
covariant derivative	$\partial u + \omega u$	$D = \nabla + \omega\mathsf{T}$	intrinsic
hat gradients	$B(B^TB)^{-1}\hat{\nabla}\hat{\phi}$	reciprocal frame, $\frac{(x_c - x_b)\mathsf{T}}{2A}$	intrinsic
stiffness block	$A(g_i \cdot g_j)R_i^T R_j$	$A(\nabla\phi_i \cdot \nabla\phi_j)e^{\mathsf{T}(\rho_j - \rho_i)}$	intrinsic up to the vertex planes
global system	real $2n \times 2n$	complex Hermitian $n \times n$	—

```
        triplets.emplace_back(ij.first, ij.second, value_ij / mass[ij.first] + (ij.
            first == ij.second ? 1.0 : 0.0));
    }
    triplets.emplace_back(source, source, 1.0);
    Eigen::SparseMatrix<Complex> S(n, n);
    S.setFromTriplets(triplets.begin(), triplets.end());
    Eigen::VectorXcd rhs = Eigen::VectorXcd::Zero(n);
    rhs(source) = value;
    Eigen::SparseLU<Eigen::SparseMatrix<Complex>> lu(S);
    return lu.solve(rhs);
}
```

18. Dictionary

Table 2 lists each object of the two parts side by side, with its nature, intrinsic or extrinsic.

The geometric algebra formulation removes the frames from the geometry: a tangent plane is a bivector, a direction in it is a complex number of that plane, and transport is a rotor. Frames reappear only as the gauge in which coordinates are stored, and the gauge covariance (15.3) guarantees that they do not affect the result. What remains of the embedding is the choice of a tangent plane at each vertex, the one extrinsic ingredient of the discrete connection, and the place where an intrinsic method would differ.

References

[1] M. P. do Carmo. *Differential Geometry of Curves and Surfaces*. Prentice-Hall, 1976.

[2] C. Doran and A. Lasenby. *Geometric Algebra for Physicists*. Cambridge University Press, 2003.

[3] D. Hestenes and G. Sobczyk. *Clifford Algebra to Geometric Calculus: A Unified Language for Mathematics and Physics*. D. Reidel, 1984.

[4] F. Knöppel, K. Crane, U. Pinkall, and P. Schröder. Globally optimal direction fields. *ACM Transactions on Graphics*, 32(4):59:1–59:10, 2013.

[5] H. P. Langtangen. Introduction to finite element methods. 2014.

[6] B. Liu, Y. Tong, F. De Goes, and M. Desbrun. Discrete connection and covariant derivative for vector field analysis and design. *ACM Transactions on Graphics*, 35(3):23:1–23:17, 2016.

[7] M. C. Lopez Belon. Project on github. Available at https://github.com/mauriciocele/basic-fem-connection-laplacian-degoes, 2024.

[8] F.-J. Sayas. A gentle introduction to the finite element method. 2015.

[9] N. Sharp, Y. Soliman, and K. Crane. The vector heat method. *ACM Transactions on Graphics*, 38(3):24:1–24:19, 2019.

Mauricio Cele Lopez Belon
Madrid, Spain
e-mail: `mclopez@outlook.com`